

MANUAL COMPLETO DE MATEMÁTICA DISCRETA

Grado en Ingeniería Informática - 1º Curso

Este documento unifica todos los temas del plan de estudio de lógica formal, teoría de números, combinatoria y teoría de grafos en un único manual para facilitar su lectura, impresión o conversión a formatos como PDF.

Índice General

- **Bloque 1: Lógica y Técnicas de Demostración**
 - [Tema 1: Lógica Proposicional](#)
 - [Tema 2: Inferencia y Deducción Lógica](#)
 - [Tema 3: Métodos de Demostración](#)
- **Bloque 2: Teoría de Números y Criptografía**
 - [Tema 4: Divisibilidad, MCD y el Algoritmo de Euclides](#)
 - [Tema 5: Aritmética Modular y Congruencias](#)
 - [Tema 6: Criptografía RSA](#)
- **Bloque 3: Combinatoria y Estructuras de Recurrencia**
 - [Tema 7: Combinatoria Básica y Principios de Recuento](#)
 - [Tema 8: Relaciones de Recurrencia Lineales](#)
 - [Tema 9: Funciones Generadoras](#)
- **Bloque 4: Teoría de Grafos y Algoritmia**
 - [Tema 10: Conceptos Básicos de Grafos y Árboles](#)
 - [Tema 11: Representación Matricial de Grafos](#)
 - [Tema 12: Algoritmos sobre Grafos](#)
- **Secciones Finales**
 - [Glosario de Términos](#)
 - [Bibliografía Recomendada](#)

Tema 1: Lógica Proposicional

La lógica proposicional es el sistema formal más simple para modelar y razonar sobre la veracidad de enunciados. Representa la base del diseño de hardware (puertas lógicas) y de la lógica condicional en la programación. Su objetivo principal es determinar si un razonamiento o argumento es formalmente válido abstrayéndose del contenido real del lenguaje natural.

1. Proposiciones y Conectores Lógicos

Una **proposición** es una declaración declarativa que es inequívocamente **verdadera** (V) o **falsa** (F), pero no ambas cosas a la vez.

- Son proposiciones*: "El número 2 es primo", "Hoy es martes", "La memoria RAM almacena datos de forma volátil".
- No son proposiciones*: "¿Qué hora es?" (pregunta), " $x + 2 = 5$ " (es una variable libre sin contexto, hasta que se defina x , es una función proposicional), "¡Cierra la puerta!" (orden).

Conectores Lógicos

Para construir proposiciones complejas a partir de proposiciones simples (atómicas), se usan conectores lógicos:

Conector	Nombre	Símbolo	Significado en lenguaje natural
Negación	NOT	$\neg P$	"No P " o "Es falso que P ".
Conjunción	AND	$P \wedge Q$	" P y Q ". Ambos deben ser verdaderos.
Disyunción	OR	$P \vee Q$	" P o Q ". Al menos uno debe ser verdadero.
Condicional	Implicación	$P \rightarrow Q$	"Si P , entonces Q ". P es antecedente, Q es consecuente.

Conector	Nombre	Símbolo	Significado en lenguaje natural
Bicondicional	Equivalencia	$P \leftrightarrow Q$	" P si y solo si Q ". Ambos deben tener el mismo valor de verdad.

2. Semántica y Tablas de Verdad

La semántica de una fórmula proposicional define su valor de verdad en función de los valores de sus variables de entrada. Esto se representa mediante una **tabla de verdad**:

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \rightarrow Q$	$P \leftrightarrow Q$
V	V	F	V	V	V	V
V	F	F	F	V	F	F
F	V	V	F	V	V	F
F	F	V	F	F	V	V

[!WARNING] **La implicación** ($P \rightarrow Q$): Es falsa únicamente cuando el antecedente P es verdadero y el consecuente Q es falso. Si P es falso, la implicación siempre es verdadera (principio de "vacuidad").

3. Tautologías, Contradicciones y Contingencias

Una proposición compuesta se clasifica según los resultados de su última columna en la tabla de verdad:

- Tautología:** Es verdadera para cualquier combinación de valores de verdad de sus variables atómicas. Se denota como **T** o 1.
- Contradicción:** Es falsa para cualquier combinación de valores de verdad. Se denota como **C** o 0.
- Contingencia:** Su valor de verdad depende de los valores de las variables (contiene tanto V como F).

4. Leyes de Equivalencia Lógica

Dos fórmulas proposicionales A y B son **lógicamente equivalentes** ($A \equiv B$) si tienen la misma tabla de verdad. Esto nos permite simplificar fórmulas proposicionales complejas algebraicamente:

- **Leyes de De Morgan:**

$$\neg(P \wedge Q) \equiv \neg P \vee \neg Q$$

$$\neg(P \vee Q) \equiv \neg P \wedge \neg Q$$

- **Ley de la Implicación (Condicional):**

$$P \rightarrow Q \equiv \neg P \vee Q$$

- **Leyes Distributivas:**

$$P \wedge (Q \vee R) \equiv (P \wedge Q) \vee (P \wedge R)$$

$$P \vee (Q \wedge R) \equiv (P \vee Q) \wedge (P \vee R)$$

- **Leyes de Absorción:**

$$P \wedge (P \vee Q) \equiv P$$

$$P \vee (P \wedge Q) \equiv P$$

5. El Toque Informático

Conexión con las Expresiones Booleanas y Evaluación en Cortocircuito

En lenguajes de programación como Python, C++ o Java, las expresiones condicionales (`if (P && Q)`) utilizan exactamente la lógica proposicional. Sin embargo, los compiladores aplican la **evaluación en cortocircuito** (short-circuit evaluation) para optimizar la ejecución:

- En $P \ \&\& \ Q$ (conjunción), si P es falso, el programa no evalúa Q (pues el resultado final será falso independientemente del valor de Q).
- En $P \ || \ Q$ (disyunción), si P es verdadero, no se evalúa Q (pues el resultado final ya es verdadero de forma segura).

Esto evita fallos en tiempo de ejecución. Por ejemplo, en `if (ptr != nullptr && ptr->val == 5)`, si el puntero `ptr` es nulo, la primera parte es falsa y la segunda parte (que daría un error de violación de acceso de puntero nulo) nunca se ejecuta.

A continuación, implementamos en Python un generador automático de tablas de verdad para evaluar expresiones proposicionales binarias.

```
# Definimos los conectores lógicos como funciones
def and_op(p, q): return p and q
def or_op(p, q): return p or q
def imp_op(p, q): return (not p) or q
def bic_op(p, q): return p == q

# Expresión a evaluar:  $\neg(P \wedge Q) \leftrightarrow (\neg P \vee \neg Q)$ 
def evaluar_expresion(p, q):
    left = not and_op(p, q)
    right = or_op(not p, not q)
    return bic_op(left, right)

# Impresión de la Tabla de verdad
print("| P | Q |  $\neg(P \wedge Q)$  |  $\neg P \vee \neg Q$  | Resultado Bicondicional |")
print("|-----|-----|-----|-----|-----|")
for p in [True, False]:
    for q in [True, False]:
        v_left = not (p and q)
        v_right = (not p) or (not q)
        res = v_left == v_right

        # Mapeamos True/False a V/F para la visualización
        to_char = lambda x: 'V' if x else 'F'
        print(f"| {to_char(p)} | {to_char(q)} | {to_char(v_left)} | {to_char(v_right)} | {to_char(res)} |")
```

6. Ejercicios Resueltos

Ejercicio 1

Demostrar mediante álgebra lógica que la expresión $(P \wedge Q) \vee (P \wedge \neg Q)$ es equivalente a P .

Solución: Aplicamos las leyes de equivalencia lógica paso a paso:

1. **Ley Distributiva** (factor común P con conjunción $\wedge$):

$$(P \wedge Q) \vee (P \wedge \neg Q) \equiv P \wedge (Q \vee \neg Q)$$

2. **Ley de la Negación (Tercero Excluido)** (sabemos que $Q \vee \neg Q$ es siempre verdadero):

$$Q \vee \neg Q \equiv \mathbf{T}$$

3. **Ley de Identidad** (cualquier proposición en conjunción con una tautología conserva su valor original):

$$P \wedge \mathbf{T} \equiv P$$

Por lo tanto, queda demostrado algebraicamente que $(P \wedge Q) \vee (P \wedge \neg Q) \equiv P$.

Ejercicio 2

Construir la tabla de verdad de la proposición compuesta $(P \rightarrow Q) \wedge (Q \rightarrow P)$ y clasificarla.

Solución: Evaluamos la expresión columna a columna para las cuatro combinaciones de variables:

1. Para $P = V, Q = V$:

$$\circ P \rightarrow Q = V, Q \rightarrow P = V \Rightarrow (V) \wedge (V) = V.$$

2. Para $P = V, Q = F$:

$$\circ P \rightarrow Q = F, Q \rightarrow P = V \Rightarrow (F) \wedge (V) = F.$$

3. Para $P = F, Q = V$:

$$\circ P \rightarrow Q = V, Q \rightarrow P = F \Rightarrow (V) \wedge (F) = F.$$

4. Para $P = F, Q = F$:

$$\circ P \rightarrow Q = V, Q \rightarrow P = V \Rightarrow (V) \wedge (V) = V.$$

La última columna contiene valores $\{V, F, F, V\}$, lo que significa que la proposición es una **contingencia** (coincide con la semántica del operador bicondicional $P \leftrightarrow Q$).

7. Ejercicios Propuestos

1. Construye la tabla de verdad de la expresión proposicional $\neg(P \vee \neg Q) \rightarrow (P \wedge Q)$ y determina si es una tautología, contradicción o contingencia.
2. Simplifica la expresión lógica $\neg(\neg P \wedge Q) \wedge (P \vee Q)$ aplicando las leyes de equivalencia proposicional detallando cada ley utilizada.
3. Escribe una expresión lógica equivalente para el operador OR exclusivo (XOR), utilizando únicamente los conectores básicos de conjunción ($\wedge$), disyunción ($\vee$) y negación ($\neg$).

Tema 2: Inferencia y Deducción Lógica

La lógica proposicional no solo sirve para evaluar fórmulas estáticas, sino para construir y validar razonamientos dinámicos. Un argumento lógico es un conjunto de premisas que pretenden justificar una conclusión. Mediante las **reglas de inferencia**, podemos realizar derivaciones formales paso a paso para verificar rigurosamente la validez de un razonamiento sin recurrir a costosas tablas de verdad.

1. El Concepto de Argumento Válido

Un **argumento** es una secuencia de proposiciones compuestas por una o más **premisas** ($P_1, P_2, \dots, P_n$) y una **conclusión** (Q). Se representa formalmente como:

$$(P_1 \wedge P_2 \wedge \dots \wedge P_n) \rightarrow Q$$

- **Validez:** Un argumento es **válido** si y solo si la implicación anterior es una tautología. En términos prácticos: si todas las premisas son verdaderas, es imposible que la conclusión sea falsa.
- **Verdad vs. Validez:** La validez es una propiedad formal de la estructura del razonamiento, no del contenido empírico. Un argumento puede ser perfectamente válido aunque sus premisas y su conclusión sean fácticamente falsas (por ejemplo: "Todos los informáticos son marcianos. Juan es informático. Por tanto, Juan es marciano").

2. Reglas de Inferencia Básicas

Las reglas de inferencia son tautologías fundamentales expresadas de forma simplificada que nos permiten deducir proposiciones nuevas (conclusiones parciales) a partir de proposiciones conocidas (premisas).

2.1 Modus Ponens (Afirmación del Antecedente)

Si una implicación y su antecedente son verdaderos, el consecuente también es verdadero:

$$\frac{P \rightarrow Q, \quad P}{Q}$$

2.2 Modus Tollens (Negación del Consecuente)

Si una implicación es verdadera y su consecuente es falso, el antecedente es falso:

$$\frac{P \rightarrow Q, \quad \neg Q}{\neg P}$$

2.3 Silogismo Hipotético (Regla de Transitividad)

Permite encadenar implicaciones:

$$\frac{P \rightarrow Q, \quad Q \rightarrow R}{P \rightarrow R}$$

2.4 Silogismo Disyuntivo

Si tenemos una disyuntiva y uno de los términos es falso, el otro debe ser verdadero:

$$\frac{P \vee Q, \quad \neg P}{Q}$$

2.5 Regla de Resolución

Es la regla fundamental en la que se basan los demostradores automáticos de teoremas en inteligencia artificial:

$$\frac{P \vee Q, \quad \neg P \vee R}{Q \vee R}$$

3. Deducción Natural y Derivaciones Formales

Una **derivación formal** consiste en escribir una lista de proposiciones numeradas, donde cada una de ellas es una premisa original o se ha obtenido aplicando una regla de inferencia a las líneas anteriores. La última línea de la derivación debe ser la conclusión deseada.

4. El Toque Informático

Programación Lógica (Prolog) y Motores de Inferencia

En ingeniería informática, las reglas de inferencia se automatizan para crear sistemas capaces de "pensar". El exponente más claro es **Prolog** (Programming in Logic), un lenguaje declarativo donde se definen hechos y reglas lógicas, y el motor de inferencia (resolución) deduce la veracidad de las consultas mediante encadenamiento hacia atrás (backtracking).

Por ejemplo, definimos en Prolog:

```
humano(juan).  
mortal(X) :- humano(X).
```

Si preguntamos `?- mortal(juan).`, Prolog busca en su base de conocimiento, aplica una unificación equivalente a un Modus Ponens y responde `true`.

A continuación, implementamos en Python un motor de inferencia simple basado en Modus Ponens para deducir nuevos hechos a partir de una lista de reglas conocidas.

```
# Base de conocimientos: hechos iniciales verdaderos  
hechos = {"compilador_optimiza", "código_bien_diseñado"}  
  
# Reglas: representadas como tuplas (antecedentes_lista, consecuente)  
# Si todos los antecedentes son verdaderos, deducimos el consecuente  
reglas = [
```

```

(["compilador_optimiza", "código_bien_diseñado"], "ejecución_rápida"),
(["ejecución_rápida"], "usuario_satisfecho")
]

print("Hechos iniciales:", hechos)
nuevo_hecho_deducido = True

# Ciclo del motor de inferencia (encadenamiento hacia adelante)
while nuevo_hecho_deducido:
    nuevo_hecho_deducido = False
    for antecedentes, consecuente in reglas:
        if consecuente not in hechos:
            # Comprobamos si todos los antecedentes de la regla están en los hechos
            conocidos
            if all(ant in hechos for ant in antecedentes):
                print(f"Deduciendo: Como {antecedentes} son verdaderos ->
{consecuente} es verdadero (Modus Ponens)")
                hechos.add(consecuente)
                nuevo_hecho_deducido = True

print("Base de conocimientos final:", hechos)

```

5. Ejercicios Resueltos

Ejercicio 1

Demostrar la validez del siguiente argumento lógico mediante deducción natural (justificando la regla y las líneas implicadas en cada paso):

- Premisa 1: $P \rightarrow Q$
- Premisa 2: $\neg Q$
- Premisa 3: $\neg P \rightarrow R$
- Conclusión: R

Solución: Escribimos la derivación formal paso a paso:

1. $P \rightarrow Q$ (Premisa)
2. $\neg Q$ (Premisa)
3. $\neg P \rightarrow R$ (Premisa)
4. $\neg P$ (Modus Tollens aplicado a las líneas 1 y 2)
5. R (Modus Ponens aplicado a las líneas 3 y 4)

La última línea coincide con la conclusión buscada (R), por lo que el razonamiento queda demostrado formalmente como **válido**.

Ejercicio 2

Analizar el siguiente razonamiento e identificar si es válido o si incurre en alguna falacia: "Si un programa tiene un bucle infinito, entonces la CPU se calienta. La CPU se calienta. Por lo tanto, el programa tiene un bucle infinito."

Solución:

1. Modelamos el argumento:

- P : El programa tiene un bucle infinito.
- Q : La CPU se calienta.
- Premisa 1: $P \rightarrow Q$
- Premisa 2: Q
- Conclusión: P

2. Evaluamos la validez formal: El razonamiento se resume como $\frac{P \rightarrow Q, Q}{P}$. Esta estructura es inválida. En lógica formal se conoce como la **Falacia de Afirmación del Consecuente**.

Físicamente, la CPU puede calentarse por múltiples razones ajenas a un bucle infinito (por ejemplo, procesamiento de renderizado 3D complejo, ventilación defectuosa o sobrecalentamiento del entorno). Por lo tanto, aunque las premisas sean verdaderas, la conclusión no es necesariamente verdadera. El razonamiento es **inválido**.

6. Ejercicios Propuestos

1. Demuestra la validez del siguiente argumento lógico mediante derivación formal paso a paso:

- Premisas: $A \rightarrow \neg B, C \rightarrow B, A \wedge D$
- Conclusión: $\neg C$

2. Dadas las premisas $P \vee Q, P \rightarrow R$, y $\neg Q$, deduce formalmente la conclusión R .

3. Investiga y explica la diferencia en lógica computacional entre el **encadenamiento hacia adelante** (forward chaining) y el **encadenamiento hacia atrás** (backward chaining) y cómo aplican estos conceptos las bases de datos deductivas.

Tema 3: Métodos de Demostración

Una demostración es un argumento lógico riguroso que establece de forma incontestable la veracidad de un teorema matemático. En las ciencias de la computación, las técnicas de demostración no solo sirven para validar enunciados matemáticos abstractos, sino para verificar formalmente que un programa de software es correcto o que un algoritmo termina y devuelve el resultado esperado para cualquier entrada posible.

1. Demostración Directa

Para demostrar una implicación $P \rightarrow Q$, la técnica más sencilla consiste en asumir que la premisa (antecedente) P es verdadera y, utilizando definiciones, axiomas y teoremas ya demostrados, deducir lógicamente que la conclusión (consecuente) Q también es verdadera.

Ejemplo

Demostrar que la suma de dos números enteros pares es par:

- Definición de par:** Un entero x es par si existe un entero k tal que $x = 2k$.
- Sean a y b dos números enteros pares. Por definición:

$$a = 2k_1, \quad b = 2k_2 \quad (\text{para ciertos enteros } k_1, k_2)$$

- Calculamos la suma:

$$a + b = 2k_1 + 2k_2 = 2(k_1 + k_2)$$

- Como la suma de dos enteros $(k_1 + k_2)$ es otro número entero k_3 , tenemos que:

$$a + b = 2k_3$$

- Por tanto, la suma $a + b$ es par. Q.E.D. (Quod erat demonstrandum).

2. Demostración por Contrarrecíproco (Contraposición)

Esta técnica se basa en la equivalencia lógica entre una implicación y su contrarrecíproco:

$$P \rightarrow Q \equiv \neg Q \rightarrow \neg P$$

Para demostrar $P \rightarrow Q$, asumimos que Q es falso ($\neg Q$) y demostramos que, bajo esa premisa, P también es falso ($\neg P$). Es de gran utilidad cuando negar el consecuente ofrece propiedades algebraicas más directas para trabajar.

3. Demostración por Contradicción (Reducción al Absurdo)

Consiste en asumir que la afirmación que deseamos demostrar (P) es falsa ($\neg P$) y derivar de ahí una contradicción o imposibilidad lógica (un enunciado del tipo $R \wedge \neg R$, como por ejemplo demostrar que un número es par e impar simultáneamente o que $0 = 1$). Si negar la hipótesis nos conduce a un absurdo inviable, la hipótesis de partida debe ser necesariamente verdadera.

4. El Principio de Inducción Matemática

Se utiliza para demostrar propiedades asociadas al conjunto de los números enteros positivos ($\mathbb{N}$). Consiste en el "efecto dominó": si demostramos que la propiedad se cumple para el primer elemento y que la veracidad de un elemento cualquiera arrastra al siguiente, entonces la propiedad es verdadera para todo el conjunto infinito de enteros.

El proceso consta de tres pasos formales:

- Paso Base:** Demostrar que la propiedad $P(n)$ es verdadera para el primer elemento (típicamente $n = 1$ o $n = 0$).

2. **Hipótesis de Inducción (H.I.):** Asumir que la propiedad es verdadera para un número entero genérico k , es decir, asumimos que $P(k)$ es válido.
3. **Paso Inductivo:** Demostrar que, asumiendo la H.I., la propiedad se cumple obligatoriamente para el elemento siguiente $k + 1$, es decir, demostrar que:

$$P(k) \rightarrow P(k + 1)$$

5. El Toque Informático

Verificación Formal de Programas y Recursión

En informática, la inducción matemática y la **recursión** son caras de la misma moneda. Una función recursiva define un caso base (paso base de la inducción) y un paso recursivo que simplifica el problema (paso inductivo).

Además, para verificar bucles iterativos se utiliza el concepto de **Invariante de Bucle (Loop Invariant)**: una condición lógica que debe cumplirse:

1. Antes de entrar al bucle (Paso Base / Inicialización).
2. En cada iteración del bucle, asumiendo que se cumplía antes (Paso Inductivo / Mantenimiento).
3. Al terminar el bucle (Finalización), permitiendo demostrar formalmente que el algoritmo ha procesado la estructura de datos correctamente sin dar lugar a errores de rango o lógica.

A continuación, implementamos en Python una comparación conceptual entre el cálculo de una potencia de forma recursiva y su verificación mediante invariantes lógicas.

```
# Algoritmo de exponenciación recursiva rápida: potencia(x, n) = x^n
# Caso base: n = 0 -> x^0 = 1
# Paso inductivo/recursivo: x^n = x * x^(n-1)
def potencia(x, n):
    assert n >= 0, "n debe ser un entero no negativo"
    if n == 0:
        return 1 # Caso Base
    else:
        return x * potencia(x, n - 1) # Paso Recursivo (Inductivo)
```



```
# Verificación de invariante mediante testing unitario inducido
x_base = 3
for exponente in range(10):
    resultado = potencia(x_base, exponente)
    resultado_esperado = x_base ** exponente

    # Comprobación de correctitud
    assert resultado == resultado_esperado, f"Error para exponente {exponente}"
    print(f"Verificado: {x_base}^{exponente} = {resultado} (Correcto)")

print("\n¡Algoritmo verificado formalmente mediante equivalencia recursiva!")
```

6. Ejercicios Resueltos

Ejercicio 1

Demostrar por contradicción (reducción al absurdo) que el número $\sqrt{2}$ es irracional.

Solución:

1. **Hipótesis de partida:** Asumimos la negación del enunciado, es decir, asumimos que $\sqrt{2}$ es un número racional.
2. Por definición de racional, podemos expresar $\sqrt{2}$ como una fracción irreducible de dos enteros a y b (donde $\text{mcd}(a, b) = 1$, es decir, no tienen factores comunes):

$$\sqrt{2} = \frac{a}{b}$$

3. Elevamos ambos lados al cuadrado:

$$2 = \frac{a^2}{b^2} \quad \Rightarrow \quad a^2 = 2b^2$$

4. Puesto que a^2 es igual a 2 por un entero (b^2), concluimos que a^2 es un número par. Si el cuadrado de un número es par, el propio número también debe ser par (se demuestra fácilmente por contraposición). Por tanto, escribimos:

$$a = 2k \quad (\text{para algún entero } k)$$

5. Sustituimos $a = 2k$ en la ecuación del paso 3:

$$(2k)^2 = 2b^2 \quad \Rightarrow \quad 4k^2 = 2b^2 \quad \Rightarrow \quad 2k^2 = b^2$$

6. Dado que $b^2 = 2k^2$, deducimos que b^2 también es par, y por ende b también debe ser par.
7. **Llegada al absurdo:** Hemos demostrado que a es par y b es par. Esto significa que ambos son divisibles por 2, lo cual entra en **contradicción directa** con la premisa inicial de que la fracción $\frac{a}{b}$ era irreducible ($\text{mcd}(a, b) = 1$).
8. Al conducir la negación a una contradicción matemática insoluble, la hipótesis del absurdo debe descartarse: por lo tanto, $\sqrt{2}$ es irracional. Q.E.D.

Ejercicio 2

Demostrar por inducción matemática que para todo entero $n \geq 1$ se cumple:

$$1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2}$$

Solución: Definimos la propiedad $P(n) \equiv \sum_{i=1}^n i = \frac{n(n+1)}{2}$.

1. **Paso Base:** Evaluamos para $n = 1$:

- Lado izquierdo: 1.
- Lado derecho: $\frac{1(1+1)}{2} = \frac{2}{2} = 1$.
- Como $1 = 1$, el paso base es verdadero.

2. **Hipótesis de Inducción (H.I.):** Asumimos que $P(k)$ es verdadero para un entero $k \geq 1$:

$$1 + 2 + 3 + \dots + k = \frac{k(k+1)}{2}$$

3. **Paso Inductivo:** Debemos demostrar que $P(k+1)$ se cumple, es decir, que:

$$1 + 2 + 3 + \dots + k + (k+1) = \frac{(k+1)((k+1)+1)}{2} = \frac{(k+1)(k+2)}{2}$$

Desarrollamos el lado izquierdo usando la H.I. para sustituir la suma de los primeros k términos:

$$\underbrace{1 + 2 + 3 + \dots + k}_{\text{Sustituimos por H.I.}} + (k+1) = \frac{k(k+1)}{2} + (k+1)$$

Sacamos factor común $(k+1)$:

$$(k+1)\left(\frac{k}{2} + 1\right) = (k+1)\left(\frac{k+2}{2}\right) = \frac{(k+1)(k+2)}{2}$$

El resultado obtenido coincide exactamente con la expresión de $P(k + 1)$. Queda demostrado por inducción que la propiedad se cumple para todo entero $n \geq 1$.

7. Ejercicios Propuestos

1. Demuestra por contrarrecíproco el siguiente enunciado sobre enteros: "Si $3n + 2$ es un número impar, entonces n es impar".
2. Demuestra por inducción matemática simple que para todo $n \geq 1$, la suma de los primeros n números impares es igual a n^2 :

$$1 + 3 + 5 + \dots + (2n - 1) = n^2$$

3. Investiga el **Principio de Inducción Fuerte** y explica en qué se diferencia del principio de inducción simple. Pon un ejemplo de aplicación (como demostrar que todo entero mayor que 1 se puede descomponer en producto de primos).

Tema 4: Divisibilidad, MCD y el Algoritmo de Euclides

La teoría de números es el estudio matemático de las propiedades de los números enteros ($\mathbb{Z}$). Aunque durante siglos se consideró una disciplina puramente teórica y abstracta, hoy en día es la columna vertebral de la seguridad informática. Comprender la divisibilidad, el cálculo eficiente del máximo común divisor (mcd) y la resolución de ecuaciones diofánticas es un prerequisite esencial para descifrar el funcionamiento de los algoritmos criptográficos modernos.

1. Divisibilidad y División Entera

Decimos que un entero a **divide** a un entero b (o que b es múltiplo de a , y se denota como $a \mid b$) si existe un número entero k tal que:

$$b = a \cdot k$$

Si a no divide a b , escribimos $a \nmid b$.

Teorema de la División Entera

Dados dos enteros a (dividendo) y b (divisor, con $b \neq 0$), existen dos únicos enteros q (cociente) y r (resto) tales que:

$$a = b \cdot q + r \quad \text{donde} \quad 0 \leq r < |b|$$

2. Máximo Común Divisor (MCD)

El **máximo común divisor** de dos enteros a y b (no ambos nulos), denotado como $mcd(a, b)$ o simplemente d , es el mayor entero que divide simultáneamente a ambos.

- Coprimidad (o primos entre sí):** Dos enteros a y b son coprimos si y solo si $mcd(a, b) = 1$.

Ineficiencia de la Factorización de Primos

El método clásico para hallar el *mcd* descomponiendo los números en sus factores primos es extremadamente lento para números grandes. Encontrar los factores primos de un entero de 1024 bits requeriría miles de años de cómputo en supercomputadoras actuales. Por ello, recurrimos al **Algoritmo de Euclides**.

3. El Algoritmo de Euclides

Este algoritmo aprovecha la siguiente propiedad recursiva de la división: si $a = b \cdot q + r$, entonces:

$$\text{mcd}(a, b) = \text{mcd}(b, r)$$

El algoritmo consiste en realizar divisiones sucesivas hasta obtener un resto igual a cero. El último resto no nulo es el $\text{mcd}(a, b)$.

```
a = b * q_1 + r_1      (mcd(a,b) = mcd(b, r_1))
b = r_1 * q_2 + r_2    (mcd(b,r_1) = mcd(r_1, r_2))
r_1 = r_2 * q_3 + 0    (El resto es cero)
--> mcd(a,b) = r_2
```

4. Algoritmo de Euclides Extendido e Identidad de Bézout

La **Identidad de Bézout** afirma que, si $d = \text{mcd}(a, b)$, existen enteros x e y tales que:

$$a \cdot x + b \cdot y = d$$

Los enteros x e y se conocen como los **coeficientes de Bézout** y pueden hallarse despejando los restos del algoritmo de Euclides en sentido inverso o mediante un esquema matricial iterativo.

5. Resolución de Ecuaciones Diofánticas Lineales

Una **ecuación diofántica lineal** es una ecuación algebraica de la forma:

$$a \cdot x + b \cdot y = c$$

donde buscamos únicamente soluciones en las que x e y sean números enteros.

Teorema de Existencia

La ecuación diofántica $ax + by = c$ tiene solución entera si y solo si el máximo común divisor de los coeficientes de entrada divide al término independiente:

$$\text{mcd}(a, b) \mid c$$

Si esta condición se cumple, podemos encontrar una solución particular (x_0, y_0) mediante el algoritmo extendido de Euclides y generalizar la solución como:

$$x = x_0 + k \cdot \frac{b}{d}, \quad y = y_0 - k \cdot \frac{a}{d} \quad (\text{para cualquier entero } k)$$

6. El Toque Informático

Complejidad del Algoritmo de Euclides y Aplicación en Criptografía

El algoritmo de Euclides es uno de los algoritmos más eficientes y antiguos conocidos.

- Complejidad Temporal:** El teorema de Lamé demuestra que el número de divisiones necesarias para calcular el mcd de dos números es, como mucho, 5 veces el número de dígitos del número menor. Su complejidad es de $O(\log(\min(a, b)))$.
- Utilidad:** En criptografía, el cálculo del algoritmo extendido de Euclides es el paso computacional fundamental utilizado para hallar el **inverso multiplicativo**

modular (indispensable para calcular la clave privada de descifrado en el algoritmo RSA).

A continuación, implementamos en Python el Algoritmo de Euclides Extendido que calcula el *mcd* y devuelve los coeficientes de la Identidad de Bézout.

```
def euclides_extendido(a, b):  
    # Caso base  
    if a == 0:  
        return b, 0, 1  
  
    # Llamada recursiva  
    mcd, x1, y1 = euclides_extendido(b % a, a)  
  
    # Actualización de los coeficientes de Bézout basados en la recursión  
    x = y1 - (b // a) * x1  
    y = x1  
  
    return mcd, x, y  
  
# Ejemplo de prueba con a = 252 y b = 198  
num1, num2 = 252, 198  
d, x, y = euclides_extendido(num1, num2)  
  
print(f"Número 1: {num1}, Número 2: {num2}")  
print(f"MCD calculado: {d}")  
print(f"Coeficientes de Bézout: x = {x}, y = {y}")  
print(f"Verificación: {num1}*({x}) + {num2}*({y}) = {num1*x + num2*y}")
```

7. Ejercicios Resueltos

Ejercicio 1

Calcular el *mcd*(252, 198) y hallar su identidad de Bézout de forma analítica (despejando restos).

Solución:

1. Algoritmo de Euclides (hacia adelante):

- $252 = 198 \cdot 1 + 54 \quad \Rightarrow \text{(resto 54)}$
- $198 = 54 \cdot 3 + 36 \quad \Rightarrow \text{(resto 36)}$
- $54 = 36 \cdot 1 + 18 \quad \Rightarrow \text{(resto 18)}$
- $36 = 18 \cdot 2 + 0 \quad \Rightarrow \text{(resto 0)}$

- El último resto no nulo es **18**. Por tanto, $\text{mcd}(252, 198) = 18$.

2. Identidad de Bézout (despeje en retroceso):

- Despejamos el 18 de la última ecuación no nula:

$$18 = 54 - 36 \cdot 1$$

- Despejamos el 36 de la ecuación previa y sustituimos:

$$36 = 198 - 54 \cdot 3$$

$$18 = 54 - (198 - 54 \cdot 3) \cdot 1 = 54 \cdot 4 - 198 \cdot 1$$

- Despejamos el 54 de la primera ecuación y sustituimos:

$$54 = 252 - 198 \cdot 1$$

$$18 = (252 - 198 \cdot 1) \cdot 4 - 198 \cdot 1 = 252 \cdot 4 - 198 \cdot 4 - 198 \cdot 1$$

$$18 = 252 \cdot (4) + 198 \cdot (-5)$$

Los coeficientes de Bézout son $x = 4$ e $y = -5$.

Ejercicio 2

Resolver la ecuación diofántica lineal: $12x + 15y = 9$.

Solución:

1. Analizar existencia de solución:

- Calculamos el $d = \text{mcd}(12, 15)$: $15 = 12 \cdot 1 + 3 \Rightarrow 12 = 3 \cdot 4 + 0 \Rightarrow \text{mcd}(12, 15) = 3$.
- Comprobamos si $d \mid c$: ¿ $3 \mid 9$? Sí ($9 = 3 \cdot 3$). **La ecuación tiene solución.**

2. Hallar coeficientes de Bézout para el MCD:

- De la división: $3 = 15 \cdot 1 + 12 \cdot (-1) \Rightarrow 12 \cdot (-1) + 15 \cdot (1) = 3$.

3. Escalar para el término independiente ($c = 9$):

- Multiplicamos la ecuación previa por 3 (pues $9 = 3 \cdot 3$):

$$12 \cdot (-3) + 15 \cdot (3) = 9$$

- Una solución particular es: $x_0 = -3$, $y_0 = 3$.

4. Expresar la solución general:

$$x = -3 + k \cdot \frac{15}{3} = -3 + 5k$$

$$y = 3 - k \cdot \frac{12}{3} = 3 - 4k \quad (\text{para cualquier } k \in \mathbb{Z})$$

8. Ejercicios Propuestos

1. Calcula el máximo común divisor de 323 y 123 mediante el algoritmo de Euclides y determina si son coprimos.
2. Halla todas las soluciones enteras de la ecuación diofántica $21x + 14y = 35$. Si no existen soluciones enteras, justifica por qué.
3. Demuestra que si a y b son enteros coprimos, entonces $\text{mcd}(a + b, a - b)$ solo puede ser igual a 1 o 2.

Tema 5: Aritmética Modular y Congruencias

La aritmética modular es un sistema aritmético para números enteros donde los números "dan la vuelta" al llegar a un cierto valor límite llamado **módulo** (m). Se le conoce coloquialmente como "aritmética del reloj". Es el pilar fundamental del álgebra computacional moderna, proporcionando estructuras matemáticas finitas donde las operaciones aritméticas son rápidas, acotadas en memoria y perfectamente reversibles (inversos modulares).

1. Relación de Congruencia

Sean $a, b \in \mathbb{Z}$ y $m \in \mathbb{Z}^+$ (con $m > 1$). Decimos que a es **congruente** con b módulo m , y lo denotamos como:

$$a \equiv b \pmod{m}$$

si y solo si la diferencia $a - b$ es divisible por m (es decir, $m \mid (a - b)$). Esto es equivalente a decir que a y b devuelven exactamente el mismo resto al dividirse entre m :

$$a \pmod{m} = b \pmod{m}$$

El Conjunto de Clases de Equivalencia $\mathbb{Z}_m$

La congruencia es una relación de equivalencia (reflexiva, simétrica y transitiva). Divide a los enteros en exactamente m clases de equivalencia representadas por el conjunto:

$$\mathbb{Z}_m = \{0, 1, 2, \dots, m-1\}$$

2. Aritmética en $\mathbb{Z}_m$ y Cálculo de Inversos

La aritmética modular conserva la suma y el producto:

1. Si $a_1 \equiv b_1 \pmod{m}$ y $a_2 \equiv b_2 \pmod{m}$, entonces:

$$(a_1 + a_2) \equiv (b_1 + b_2) \pmod{m}$$

$$(a_1 \cdot a_2) \equiv (b_1 \cdot b_2) \pmod{m}$$

El Inverso Modular

En la aritmética real, el inverso de a es $1/a$. En aritmética modular, no existen los números decimales. Definimos el **inverso modular** de a módulo m como un entero $x \in \mathbb{Z}_m$ tal que:

$$a \cdot x \equiv 1 \pmod{m}$$

- **Condición de Existencia:** El inverso de a módulo m existe si y solo si a y m son coprimos:

$$\text{mcd}(a, m) = 1$$

- **Cálculo:** Se calcula aplicando el Algoritmo de Euclides Extendido. Obtenemos los coeficientes $a \cdot x + m \cdot y = 1$. Tomando módulo m en ambos lados:

$$a \cdot x \equiv 1 \pmod{m}$$

Por tanto, el coeficiente x de Bézout (ajustado para ser positivo sumándole m si es negativo) es el inverso modular buscado.

3. Teoremas Fundamentales

3.1 Teorema del Resto Chino (CRT)

Si tenemos un sistema de congruencias lineales con módulos coprimos dos a dos:

$$x \equiv a_1 \pmod{m_1}$$

$$x \equiv a_2 \pmod{m_2}$$

...

$$x \equiv a_k \pmod{m_k}$$

El teorema garantiza que existe una solución única para x módulo $M = m_1 \cdot m_2 \dots m_k$.

3.2 Pequeño Teorema de Fermat

Si p es un número primo y a es un entero tal que $p \nmid a$, entonces:

$$a^{p-1} \equiv 1 \pmod{p}$$

Multiplicando por a , se obtiene la forma general válida para cualquier entero a :

$$a^p \equiv a \pmod{p}$$

3.3 Función ϕ de Euler y Teorema de Euler

La **función indicatriz de Euler** $\phi(n)$ define el número de enteros entre 1 y n que son coprimos con n .

- Si p es primo: $\phi(p) = p - 1$.
- Si $n = p \cdot q$ (con p y q primos distintos): $\phi(n) = (p - 1)(q - 1)$.
- **Teorema de Euler:** Si a y n son coprimos ($\text{mcd}(a, n) = 1$), entonces:

$$a^{\phi(n)} \equiv 1 \pmod{n}$$

4. El Toque Informático

Algoritmos de Hashing, Checksums y Códigos de Control

La aritmética modular se utiliza para garantizar la integridad y distribución de los datos en informática:

1. **Dígitos de Control (DNI, Tarjetas de Crédito, IBAN):** El número de control del DNI en España se calcula tomando la parte numérica módulo 23: $\text{DNI} \pmod{23}$. El resto resultante se mapea con una letra específica. Si se introduce mal un

dígito del número, el módulo no coincidirá y el sistema detectará el error de entrada instantáneamente.

2. **Tablas Hash:** Para almacenar claves de forma uniforme en memoria minimizando colisiones, las funciones hash indexan elementos haciendo:

$$\text{índice} = \text{clave} \pmod{\text{tamaño_tabla}}$$

Elegir un tamaño de tabla que sea un número primo minimiza las colisiones cuando las claves siguen patrones lógicos.

A continuación, implementamos en Python un calculador de inversos modulares utilizando el algoritmo de Euclides extendido.

```
def modular_inverse(a, m):
    # Función auxiliar del algoritmo extendido
    def gcd_extended(a, b):
        if a == 0:
            return b, 0, 1
        g, x1, y1 = gcd_extended(b % a, a)
        return g, y1 - (b // a) * x1, x1

    g, x, y = gcd_extended(a, m)
    if g != 1:
        # Si el MCD no es 1, no existe inverso modular
        return None
    else:
        # Nos aseguramos de que el resultado sea positivo en Z_m
        return x % m

# Parámetros de prueba
a = 7
modulo = 26
inverso = modular_inverse(a, modulo)

if inverso is not None:
    print(f"El inverso de {a} mod {modulo} es: {inverso}")
    print(f"Verificación: ({a} * {inverso}) mod {modulo} = {(a * inverso) % modulo}")
else:
    print(f"No existe inverso modular para {a} mod {modulo} (no son coprimos)")
```

5. Ejercicios Resueltos

Ejercicio 1

Calcular el inverso modular de 7 módulo 26 de forma analítica.

Solución:

1. **Verificar coprimidad:** Aplicamos Euclides para comprobar si $\text{mcd}(7, 26) = 1$:

- $26 = 7 \cdot 3 + 5$
- $7 = 5 \cdot 1 + 2$
- $5 = 2 \cdot 2 + 1 \implies \text{mcd}(7, 26) = 1$. **Existe inverso.**

2. **Despejar restos (Bézout):**

- $1 = 5 - 2 \cdot 2$
- Sustituimos $2 = 7 - 5 \cdot 1$:

$$1 = 5 - (7 - 5 \cdot 1) \cdot 2 = 5 \cdot 3 - 7 \cdot 2$$

- Sustituimos $5 = 26 - 7 \cdot 3$:

$$1 = (26 - 7 \cdot 3) \cdot 3 - 7 \cdot 2 = 26 \cdot 3 - 7 \cdot 9 - 7 \cdot 2$$

$$1 = 26 \cdot (3) + 7 \cdot (-11)$$

3. **Aplicar congruencia módulo 26:**

$$7 \cdot (-11) \equiv 1 \pmod{26}$$

El inverso es $-11 \pmod{26}$. Lo pasamos a positivo sumándole el módulo:

$$-11 + 26 = 15$$

Por tanto, el inverso modular de 7 $\pmod{26}$ es **15**.

Ejercicio 2

Resolver el sistema de congruencias lineales:

$$x \equiv 2 \pmod{3}$$

$$x \equiv 3 \pmod{5}$$

Solución: Aplicamos el Teorema del Resto Chino:

1. **Calcular el módulo total:** $M = m_1 \cdot m_2 = 3 \cdot 5 = 15$.

2. **Calcular componentes parciales:**

- $M_1 = M/m_1 = 15/3 = 5$.
- $M_2 = M/m_2 = 15/5 = 3$.

3. Calcular inversos modulares de M_i :

- Para M_1 : $5 \cdot y_1 \equiv 1 \pmod{3} \Rightarrow 2 \cdot y_1 \equiv 1 \pmod{3} \Rightarrow y_1 = 2$.
- Para M_2 : $3 \cdot y_2 \equiv 1 \pmod{5} \Rightarrow y_2 = 2$ (pues $3 \cdot 2 = 6 \equiv 1 \pmod{5}$).

4. Construir la solución única:

$$x = (a_1 \cdot M_1 \cdot y_1 + a_2 \cdot M_2 \cdot y_2) \pmod{M}$$

$$x = (2 \cdot 5 \cdot 2 + 3 \cdot 3 \cdot 2) \pmod{15}$$

$$x = (20 + 18) \pmod{15} = 38 \pmod{15} = 8$$

La solución única al sistema de congruencias es $x \equiv 8 \pmod{15}$.

6. Ejercicios Propuestos

1. Determina si existe el inverso de $12 \pmod{30}$. Si existe, calcúlalo; si no existe, justifica algebraicamente la respuesta.
2. Resuelve el sistema de congruencias mediante el Teorema del Resto Chino:

$$x \equiv 1 \pmod{5}, \quad x \equiv 2 \pmod{7}$$

3. Utiliza el Pequeño Teorema de Fermat para simplificar y calcular rápidamente el valor de la potencia gigante $3^{202} \pmod{101}$ sin realizar multiplicaciones iteradas.

Tema 6: Criptografía RSA

La criptografía de clave pública o asimétrica resolvió uno de los mayores problemas históricos de las comunicaciones: ¿cómo pueden dos partes intercambiar información de forma segura sin haberse transmitido previamente una clave secreta compartida? El algoritmo **RSA** (desarrollado por Rivest, Shamir y Adleman en 1977) se basa en una asimetría matemática simple: multiplicar dos números primos grandes es computacionalmente fácil y rápido, pero deshacer la operación (factorizar el producto de vuelta en sus componentes primos) es extremadamente difícil.

1. El Concepto de Criptografía Asimétrica

A diferencia de la criptografía simétrica (como AES, donde se usa la misma clave para cifrar y descifrar), en la criptografía asimétrica cada participante genera un par de claves:

- Clave Pública:** Se difunde abiertamente al mundo. Cualquiera puede usar esta clave para cifrar un mensaje destinado a ti.
 - Clave Privada:** Se mantiene en estricto secreto. Solo tú dispones de ella y es la única que puede descifrar los mensajes cifrados con tu clave pública.
-

2. El Algoritmo RSA: Generación de Claves

El proceso matemático para generar el par de claves RSA se realiza siguiendo estos pasos:

- Selección de primos:** Se eligen dos números primos gigantescos distintos, p y q (en la práctica, de más de 1024 o 2048 bits de longitud cada uno).
- Módulo de cifrado:** Se calcula el producto de ambos primos:

$$n = p \cdot q$$

El número n se hace público y define el tamaño del espacio de claves.

3. **Función indicatriz:** Se calcula el valor de la función $\phi(n)$ de Euler:

$$\phi(n) = (p - 1)(q - 1)$$

Este valor se mantiene en absoluto secreto.

4. **Exponente público (e):** Se escoge un entero e (exponente de cifrado) tal que sea menor y coprimo con $\phi(n)$:

$$1 < e < \phi(n) \quad \text{y} \quad \text{mcd}(e, \phi(n)) = 1$$

Un valor común estándar en la industria es $e = 65537 (2^{16} + 1)$, por su eficiencia de cálculo.

5. **Exponente privado (d):** Se calcula el exponente de descifrado d como el único inverso modular de e módulo $\phi(n)$:

$$d \cdot e \equiv 1 \pmod{\phi(n)}$$

Este inverso se calcula de forma instantánea usando el Algoritmo de Euclides Extendido.

- **Clave Pública:** Constituida por el par (e, n) .
- **Clave Privada:** Constituida por el par (d, n) (los factores primos originales p y q y el valor $\phi(n)$ deben ser destruidos de forma segura una vez terminado el proceso).

3. Cifrado y Descifrado RSA

Para transmitir un mensaje de texto, se traduce primero a un número entero M (por ejemplo, utilizando su representación binaria/ASCII) tal que $0 \leq M < n$.

Cifrado

El emisor cifra el mensaje M utilizando la clave pública del receptor (e, n) , obteniendo el criptograma C :

$$C = M^e \pmod{n}$$

Descifrado

El receptor descifra el criptograma C utilizando su clave privada (d, n) , recuperando el mensaje original M :

$$M = C^d \pmod{n}$$

Justificación matemática: Según el teorema de Euler, como $d \cdot e \equiv 1 \pmod{\phi(n)}$, existe un entero k tal que $d \cdot e = k \cdot \phi(n) + 1$. Por tanto:

$$C^d \equiv (M^e)^d \equiv M^{e \cdot d} \equiv M^{k \cdot \phi(n) + 1} \equiv (M^{\phi(n)})^k \cdot M^1 \equiv (1)^k \cdot M \equiv M \pmod{n}$$

4. Algoritmo de Exponenciación Modular Rápida

Calcular $M^e \pmod{n}$ cuando M, e, n tienen miles de bits es inviable si se multiplican secuencialmente. En su lugar se utiliza el método de **exponenciación rápida** (Square-and-Multiply):

1. Se expresa el exponente e en formato binario.
2. Se procesa el exponente bit a bit de izquierda a derecha. En cada paso se eleva al cuadrado el acumulador actual módulo n .
3. Si el bit analizado es un 1, además se multiplica el acumulador por la base M módulo n .

Este algoritmo reduce el coste de cálculo de $O(e)$ multiplicaciones a solo $O(\log e)$ operaciones de multiplicación y módulo.

5. El Toque Informático

Infraestructura de Clave Pública (PKI), Certificados SSL y SSH

RSA es la base que permite establecer conexiones seguras en Internet:

- **HTTPS (SSL/TLS):** Cuando te conectas a tu banco, tu navegador utiliza criptografía asimétrica (como RSA o curvas elípticas) para autenticar la identidad del servidor del banco y acordar de forma segura una clave temporal simétrica (mucho más rápida) para cifrar los datos de la sesión.
- **Llaves SSH:** Los desarrolladores y administradores de sistemas utilizan llaves públicas y privadas RSA para iniciar sesión en servidores remotos de forma segura sin necesidad de contraseñas vulnerables a ataques de fuerza bruta.

A continuación, implementamos en Python una simulación completa de RSA: generación de claves con números primos pequeños, cifrado de un entero y descifrado.

```
# Función para calcular el MCD de forma recursiva
def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

# Algoritmo de Euclides Extendido para hallar el inverso modular
def mod_inverse(e, phi):
    def gcd_extended(a, b):
        if a == 0:
            return b, 0, 1
        g, x1, y1 = gcd_extended(b % a, a)
        return g, y1 - (b // a) * x1, x1

    g, x, y = gcd_extended(e, phi)
    return x % phi

# Simulación de Exponenciación Modular Rápida (Square-and-Multiply)
def exp_modular(base, exp, mod):
    res = 1
    base = base % mod
    while exp > 0:
        # Si el bit más a la derecha es 1 (impar)
        if (exp % 2) == 1:
            res = (res * base) % mod
        # Desplazamos exponente e indicamos cuadrado
        exp = exp // 2
        base = (base * base) % mod
    return res

# 1. Generación de claves con primos pequeños
p, q = 61, 53          # Primos de prueba
n = p * q              # Módulo: 3233
phi = (p - 1) * (q - 1) # phi(n): 3120

# Escogemos un exponente e coprimo con 3120
e = 17
assert gcd(e, phi) == 1, "e debe ser coprimo con phi"
```

```
# Calculamos d (inverso modular de e)
d = mod_inverse(e, phi) # d = 2753

# 2. Cifrado
mensaje_original = 65 # Carácter 'A' en ASCII
criptograma = exp_modular(mensaje_original, e, n)

# 3. Descifrado
mensaje_descifrado = exp_modular(criptograma, d, n)

print(f"Módulo n: {n}, Función phi(n): {phi}")
print(f"Clave Pública (e, n): ({e}, {n})")
print(f"Clave Privada (d, n): ({d}, {n})")
print(f"Mensaje original: {mensaje_original}")
print(f"Mensaje cifrado (criptograma enviado por la red): {criptograma}")
print(f"Mensaje descifrado: {mensaje_descifrado}")
```

6. Ejercicios Resueltos

Ejercicio 1

Simular el algoritmo RSA a mano para los siguientes parámetros: primos $p = 3$, $q = 11$, y exponente de cifrado $e = 3$. Cifrar y descifrar el mensaje $M = 4$.

Solución:

1. Cálculo de parámetros:

- $n = p \cdot q = 3 \cdot 11 = 33$.
- $\phi(n) = (p - 1)(q - 1) = 2 \cdot 10 = 20$.

2. Calcular el exponente privado d :

- Queremos $d \cdot e \equiv 1 \pmod{\phi(n)} \Rightarrow 3 \cdot d \equiv 1 \pmod{20}$.
- Probamos múltiplos: $3 \cdot d = 21 \equiv 1 \pmod{20} \Rightarrow d = 7$.

3. Cifrar el mensaje $M = 4$:

- $C = M^e \pmod{n} = 4^3 \pmod{33} = 64 \pmod{33}$.
- $64 = 33 \cdot 1 + 31 \Rightarrow C = 31$.

4. Descifrar el criptograma $C = 31$:

- $M = C^d \pmod{n} = 31^7 \pmod{33}$.
- Dado que $31 \equiv -2 \pmod{33}$ (para simplificar los cálculos a mano):

$$31^7 \equiv (-2)^7 \equiv -128 \pmod{33}$$

- Buscamos la congruencia positiva para -128 :

$$-128 = 33 \cdot (-4) + 4 \Rightarrow M = 4$$

El mensaje descifrado coincide con el mensaje original, verificando la corrección del proceso.

Ejercicio 2

Calcular de forma eficiente $5^{13} \pmod{11}$ utilizando el algoritmo de exponenciación rápida.

Solución:

1. **Expresar el exponente en binario:**

- 13 en binario es 1101_2 ($13 = 8 + 4 + 1$).

2. **Ejecutar el algoritmo paso a paso** (inicializamos el acumulador $R = 1$ y la base actual $B = 5$):

- Bit 1 (peso 1, a la derecha): es 1 $\Rightarrow R = (R \cdot B) \pmod{11} = (1 \cdot 5) \pmod{11} = 5$.
 - Elevamos la base al cuadrado: $B = B^2 \pmod{11} = 25 \pmod{11} = 3$.
- Bit 2 (peso 2): es 0 $\Rightarrow R$ no cambia ($R = 5$).
 - Elevamos la base al cuadrado: $B = B^2 \pmod{11} = 3^2 \pmod{11} = 9$.
- Bit 3 (peso 4): es 1 $\Rightarrow R = (R \cdot B) \pmod{11} = (5 \cdot 9) \pmod{11} = 45 \pmod{11} = 1$.
 - Elevamos la base al cuadrado: $B = B^2 \pmod{11} = 9^2 \pmod{11} = 81 \pmod{11} = 4$.
- Bit 4 (peso 8): es 1 $\Rightarrow R = (R \cdot B) \pmod{11} = (1 \cdot 4) \pmod{11} = 4$.

Por lo tanto, $5^{13} \equiv 4 \pmod{11}$.

7. Ejercicios Propuestos

1. Dada la clave pública RSA ($e = 7, n = 55$), calcula la clave privada correspondiente (el exponente d) sabiendo que n se descompone en los primos $p = 5$ y $q = 11$.

2. Cifra el mensaje $M = 2$ utilizando la clave pública del ejercicio anterior y descífralo paso a paso para comprobar el funcionamiento.
3. Explica por qué es fundamental que un atacante no pueda conocer el valor de la función $\phi(n)$ y cómo, si lograra factorizar el módulo n en sus componentes p y q , podría romper inmediatamente la seguridad del cifrado RSA.

Tema 7: Combinatoria Básica y Principios de Recuento

La combinatoria es el arte y la ciencia de contar. En ingeniería informática, determinar el número de elementos de un conjunto discreto o el número de formas en que puede ocurrir un evento es esencial para evaluar el consumo de memoria de una estructura de datos, calcular la probabilidad de fallos y estimar el tiempo de ejecución de algoritmos exhaustivos (análisis de complejidad temporal).

1. Principios Fundamentales del Recuento

Toda la combinatoria se apoya en dos reglas lógicas fundamentales:

- Principio de la Adición (Regla del "O"):** Si un evento A puede ocurrir de n maneras distintas y un evento B de m maneras, y ambos eventos **no pueden ocurrir simultáneamente** (son mutuamente excluyentes), entonces el número de formas en que puede ocurrir A o B es:

$$\text{Total} = n + m$$

- Principio de la Multiplicación (Regla del "Y"):** Si un experimento consta de dos etapas sucesivas, donde la primera etapa puede tener n resultados distintos y la segunda etapa m resultados, el número total de formas en que pueden ocurrir ambas etapas de forma consecutiva es:

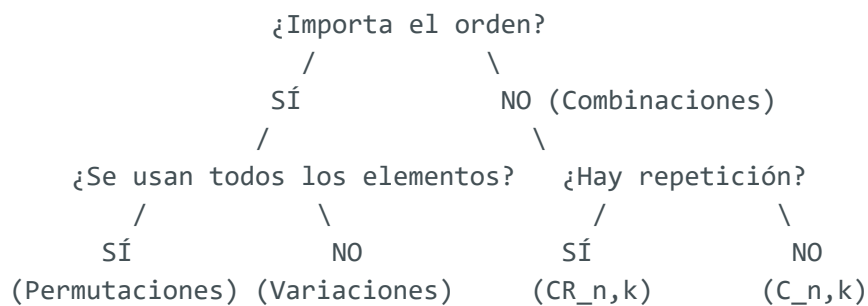
$$\text{Total} = n \cdot m$$

2. Técnicas de Recuento Clásicas

Para elegir la fórmula adecuada al resolver un problema de recuento, debemos hacernos dos preguntas fundamentales:

- ¿**Importa el orden** en el que colocamos los elementos?

2. ¿Se pueden **repetir** los elementos?



2.1 Variaciones (El orden importa; NO se usan todos los elementos)

- **Sin repetición:** Formas de elegir y ordenar k elementos de un conjunto de n :

$$V_{n,k} = \frac{n!}{(n-k)!}$$

- **Con repetición:**

$$V R_{n,k} = n^k$$

2.2 Permutaciones (El orden importa; SÍ se usan todos los elementos)

- **Sin repetición:** Formas de ordenar n elementos distintos:

$$P_n = n!$$

- **Con repetición:** Cuando algunos elementos están repetidos (a veces el primero, b el segundo...):

$$P R_n^{a,b,c,\dots} = \frac{n!}{a! \cdot b! \cdot c! \dots}$$

2.3 Combinaciones (El orden NO importa)

- **Sin repetición:** Formas de seleccionar un subgrupo de k elementos de un conjunto de n :

$$C_{n,k} = \binom{n}{k} = \frac{n!}{k!(n-k)!}$$

- **Con repetición:** Formas de seleccionar k elementos de un conjunto de n pudiendo repetir elementos:

$$CR_{n,k} = \binom{n+k-1}{k} = \frac{(n+k-1)!}{k!(n-1)!}$$

3. El Principio del Palomar (Dirichlet)

El **Principio del Palomar** afirma que si distribuimos n objetos (palomas) en m contenedores (nidos), y $n > m$, entonces **al menos un contenedor** debe albergar más de un objeto.

- **Formulación General:** Si n objetos se colocan en m cajas, entonces al menos una caja contendrá al menos $\lceil n/m \rceil$ objetos (donde $\lceil x \rceil$ representa la función techo, el menor entero mayor o igual que x).
 - **Utilidad:** Aunque parezca obvio, se utiliza para demostrar la existencia de límites o patrones en teoría de grafos, algoritmos de compresión y tablas hash.
-

4. El Toque Informático

Entropía de Contraseñas y Explosión Combinatoria

La seguridad de las contraseñas se rige por las variaciones con repetición.

Supongamos que definimos una contraseña de longitud L :

- Si solo usamos dígitos (10 caracteres posibles), el espacio de búsqueda para un atacante por fuerza bruta es de 10^L .
- Si usamos minúsculas, mayúsculas y dígitos (62 caracteres posibles), el espacio crece de forma exponencial a 62^L .

Por ejemplo, para una contraseña de 8 caracteres:

- Solo dígitos: $10^8 = 100.000.000$ combinaciones (se rompe en milisegundos).
- Alfanumérica: $62^8 \approx 2.18 \times 10^{14}$ combinaciones (requiere meses o años de cómputo en hardware estándar).

Este crecimiento exponencial se conoce como **explosión combinatoria** y es la razón por la cual los algoritmos de complejidad exponencial $O(2^n)$ o $O(n!)$ (como el del viajante de comercio por fuerza bruta) son computacionalmente inviables para valores de $n > 20$.

A continuación, implementamos en Python una utilidad que calcula factoriales y coeficientes combinatorios, demostrando la rapidez de la explosión combinatoria.

```
import math

# Función para calcular combinaciones sin repetición C(n, k)
def combinaciones(n, k):
    if k < 0 or k > n:
        return 0
    return math.factorial(n) // (math.factorial(k) * math.factorial(n - k))

# Simulación: Crecimiento de factoriales (Explosión Combinatoria)
print("Ejemplo de Explosión Combinatoria (n!):")
for i in [5, 10, 15, 20]:
    print(f" {i:2d}! = {math.factorial(i):20d}")

# Cálculo de combinaciones posibles en una mano de poker (5 cartas de un mazo de 52)
n_cartas = 52
k_mano = 5
poker_comb = combinaciones(n_cartas, k_mano)
print(f"\nCombinaciones distintas para una mano de póker (C({n_cartas}, {k_mano})): {poker_comb}")
```

5. Ejercicios Resueltos

Ejercicio 1

En una asignatura de Ingeniería Informática hay 8 estudiantes de primer curso y 6 de segundo curso. Se desea formar un grupo de trabajo compuesto por 3 estudiantes de primero y 2 de segundo. ¿Cuántos grupos distintos se pueden formar?

Solución:

1. Analizar las etapas del recuento:

- Etapa 1: Elegir 3 estudiantes de primero de un total de 8. El orden de elección no importa (es un subgrupo), por lo que usamos combinaciones sin repetición:

$$C_{8,3} = \binom{8}{3} = \frac{8!}{3! \cdot 5!} = \frac{8 \cdot 7 \cdot 6}{3 \cdot 2 \cdot 1} = 56$$

- Etapa 2: Elegir 2 estudiantes de segundo de un total de 6. Igualmente, usamos combinaciones sin repetición:

$$C_{6,2} = \binom{6}{2} = \frac{6!}{2! \cdot 4!} = \frac{6 \cdot 5}{2 \cdot 1} = 15$$

- ## 2. Combinar las etapas (Principio de la Multiplicación):
- Como debemos formar el grupo eligiendo estudiantes de primero **y** estudiantes de segundo:

$$\text{Total} = C_{8,3} \cdot C_{6,2} = 56 \cdot 15 = 840$$

Se pueden formar exactamente 840 grupos de trabajo distintos.

Ejercicio 2

Demostrar mediante el Principio del Palomar que en cualquier conjunto de 8 enteros elegidos al azar, al menos dos de ellos deben devolver el mismo resto al dividirse entre 7.

Solución:

1. **Identificar los elementos (palomas):** Los objetos a distribuir son los 8 números enteros elegidos al azar ($n = 8$).
 2. **Identificar los contenedores (nidos):** Las cajas representan los posibles restos que se obtienen al realizar una división entera entre 7. Según el teorema de la división, el resto r satisface $0 \leq r < 7$. Por lo tanto, hay exactamente 7 restos posibles: $\{0, 1, 2, 3, 4, 5, 6\}$, por lo que tenemos $m = 7$ cajas.
 3. **Aplicar el Principio del Palomar:** Dado que tenemos $n = 8$ enteros (palomas) y $m = 7$ posibles restos (nidos), y $8 > 7$, el Principio del Palomar nos garantiza que al menos dos enteros deben ser colocados en el mismo contenedor, es decir, **deben devolver el mismo resto módulo 7.**
-

6. Ejercicios Propuestos

1. ¿Cuántas palabras de 5 letras (tengan o no sentido en castellano) se pueden formar utilizando las letras de la palabra "NODO"? Justifica qué tipo de estructura combinatoria estás usando.
2. Un servidor de correo electrónico recibe 100 correos en un intervalo de un minuto. Demuestra que al menos dos correos tuvieron que haber llegado exactamente en el mismo segundo de ese minuto.
3. Calcula el número de formas en que un programador puede distribuir 8 tareas idénticas entre 3 servidores distintos si algunos servidores pueden quedarse vacíos (pista: combinaciones con repetición).

Tema 8: Relaciones de Recurrencia Lineales

Una relación de recurrencia es una ecuación que define una sucesión matemática de forma recursiva, es decir, el término actual a_n se expresa en función de uno o más términos anteriores (como a_{n-1} , a_{n-2} , etc.). En ingeniería informática, resolver una relación de recurrencia consiste en hallar una "fórmula cerrada" que nos permita calcular directamente el término a_n para cualquier n sin tener que computar todos los términos anteriores. Esto es fundamental para analizar el coste computacional de algoritmos recursivos.

1. Definición y Clasificación

Una relación de recurrencia lineal de orden k con coeficientes constantes tiene la forma:

$$a_n = c_1 a_{n-1} + c_2 a_{n-2} + \dots + c_k a_{n-k} + F(n)$$

donde $c_1, \dots, c_k$ son constantes reales ($c_k \neq 0$) y $F(n)$ es una función de n .

- Orden (k):** El número de términos anteriores de los que depende a_n .
- Homogénea:** Si $F(n) = 0$ para todo n .
- No Homogénea:** Si $F(n) \neq 0$.
- Condiciones Iniciales:** Para hallar una solución única, necesitamos conocer los primeros k valores de la sucesión $(a_0, a_1, \dots, a_{k-1})$.

2. Resolución de Recurrencias Lineales Homogéneas

Para resolver la relación homogénea de orden 2:

$$a_n - c_1 a_{n-1} - c_2 a_{n-2} = 0$$

Proponemos una solución de la forma $a_n = r^n$. Sustituyendo y dividiendo por r^{n-2} obtenemos el **polinomio característico**:

$$r^2 - c_1 r - c_2 = 0$$

Calculamos las raíces de este polinomio. Según la naturaleza de las raíces:

Caso 1: Raíces Reales y Distintas ($r_1 \neq r_2$)

La solución general es una combinación lineal de las potencias de las raíces:

$$a_n = \alpha_1 r_1^n + \alpha_2 r_2^n$$

Caso 2: Raíz Real Múltiple o Doble ($r_1 = r_2 = r$)

La solución general añade un factor multiplicador lineal n :

$$a_n = (\alpha_1 + \alpha_2 n) r^n$$

Los coeficientes α_1 y α_2 se determinan sustituyendo las condiciones iniciales de a_0 y a_1 .

3. Resolución de Recurrencias Lineales No Homogéneas

La solución general de la ecuación no homogénea $a_n = c_1 a_{n-1} + c_2 a_{n-2} + F(n)$ se compone de la suma de dos partes:

$$a_n = a_n^{(h)} + a_n^{(p)}$$

donde:

- $a_n^{(h)}$: Solución general de la relación homogénea asociada (haciendo $F(n) = 0$).
- $a_n^{(p)}$: Solución particular que tiene una forma similar a la función forzante $F(n)$.

Método de Coeficientes Indeterminados para la Solución Particular $a_n^{(p)}$

Si $F(n)$ es de la forma:	Proponemos $a_n^{(p)}$ de la forma:
Polinomio de grado d ($An^d + \dots$)	$n^s(p_d n^d + \dots + p_0)$
Exponencial ($A \cdot m^n$)	$n^s(p_0 \cdot m^n)$

(Donde s es la multiplicidad de la base de la exponencial o la raíz 1 en el polinomio característico de la parte homogénea, para evitar redundancias).

4. El Toque Informático

Coste de Algoritmos Recursivos y la Trampa de Fibonacci

El análisis de algoritmos divide el coste temporal de un programa recursivo en una relación de recurrencia:

- **Divide y Vencerás:** El algoritmo de ordenación Merge Sort divide el problema en dos subproblemas de tamaño $n/2$ y realiza un trabajo lineal $O(n)$ para mezclarlos. Su recurrencia es:

$$T(n) = 2T(n/2) + n \quad \Rightarrow \quad T(n) \in O(n \log n)$$

- **La trampa de la recursión ineficiente:** La definición de Fibonacci ($F_n = F_{n-1} + F_{n-2}$) programada de forma recursiva directa genera un árbol de llamadas exponencial ($O(1.618^n)$). Resolver este problema requiere usar **Programación Dinámica** para almacenar los resultados intermedios y reducir la complejidad a $O(n)$ lineal.

A continuación, implementamos en Python una comparación empírica de tiempo entre el cálculo de Fibonacci recursivo ingenuo y el método iterativo lineal.

```
import time
```

```

# Fibonacci recursivo ingenuo:  $O(2^n)$  o  $O(1.618^n)$ 
def fib_recursivo(n):
    if n <= 1:
        return n
    return fib_recursivo(n-1) + fib_recursivo(n-2)

# Fibonacci iterativo (Programación Dinámica):  $O(n)$ 
def fib_iterativo(n):
    if n <= 1:
        return n
    a, b = 0, 1
    for _ in range(2, n + 1):
        a, b = b, a + b
    return b

# Medición de tiempos
n_test = 30

start = time.time()
res_rec = fib_recursivo(n_test)
time_rec = time.time() - start

start = time.time()
res_it = fib_iterativo(n_test)
time_it = time.time() - start

print(f"Fibonacci({n_test}) = {res_it}")
print(f"Tiempo Recursivo Ingenuo: {time_rec:.6f} segundos")
print(f"Tiempo Iterativo (Dinámico): {time_it:.6f} segundos")
print(f"El método iterativo es {time_rec / max(time_it, 1e-9):.1f} veces más rápido.")

```

5. Ejercicios Resueltos

Ejercicio 1

Resolver la relación de recurrencia homogénea de segundo orden:

$$a_n = 5a_{n-1} - 6a_{n-2} \quad (\text{para } n \geq 2)$$

con las condiciones iniciales $a_0 = 1$ y $a_1 = 4$.

Solución:

1. Formular la ecuación característica:

$$r^2 - 5r + 6 = 0$$

2. Calcular las raíces:

$$(r-2)(r-3) = 0 \implies r_1 = 2, \quad r_2 = 3$$

Puesto que las raíces son reales y distintas, la solución general es:

$$a_n = \alpha_1 \cdot 2^n + \alpha_2 \cdot 3^n$$

3. Aplicar condiciones iniciales:

- Para $n = 0$: $\alpha_1 \cdot 2^0 + \alpha_2 \cdot 3^0 = 1 \implies \alpha_1 + \alpha_2 = 1$
- Para $n = 1$: $\alpha_1 \cdot 2^1 + \alpha_2 \cdot 3^1 = 4 \implies 2\alpha_1 + 3\alpha_2 = 4$

4. Resolver el sistema de ecuaciones:

- De la primera ecuación: $\alpha_1 = 1 - \alpha_2$.
- Sustituyendo en la segunda: $2(1 - \alpha_2) + 3\alpha_2 = 4 \implies 2 - 2\alpha_2 + 3\alpha_2 = 4 \implies \alpha_2 = 2$.
- Por lo tanto: $\alpha_1 = 1 - 2 = -1$.

5. Fórmula cerrada final:

$$a_n = -2^n + 2 \cdot 3^n \quad (\text{para } n \geq 0)$$

Ejercicio 2

Resolver la relación de recurrencia no homogénea:

$$a_n = 2a_{n-1} + 3^n$$

con condición inicial $a_0 = 1$.

Solución:

1. Resolver la parte homogénea ($a_n = 2a_{n-1}$):

- Ecuación característica: $r - 2 = 0 \implies r = 2$.
- Solución homogénea: $a_n^{(h)} = \alpha \cdot 2^n$.

2. Hallar la solución particular $a_n^{(p)}$:

- La parte no homogénea es $F(n) = 3^n$. La base es 3, que no es raíz del polinomio característico (el cual es 2).
- Proponemos: $a_n^{(p)} = P_0 \cdot 3^n$.
- Sustituyendo en la relación no homogénea original:

$$P_0 \cdot 3^n = 2(P_0 \cdot 3^{n-1}) + 3^n$$

- Dividimos por 3^{n-1} :

$$3P_0 = 2P_0 + 3 \quad \Rightarrow \quad P_0 = 3$$

- Por tanto, la solución particular es: $a_n^{(p)} = 3 \cdot 3^n = 3^{n+1}$.

3. Combinar soluciones:

$$a_n = a_n^{(h)} + a_n^{(p)} = \alpha \cdot 2^n + 3^{n+1}$$

4. Aplicar condición inicial $a_0 = 1$:

$$a_0 = \alpha \cdot 2^0 + 3^{0+1} = 1 \Rightarrow \alpha + 3 = 1 \Rightarrow \alpha = -2$$

5. Fórmula cerrada final:

$$a_n = -2 \cdot 2^n + 3^{n+1} = -2^{n+1} + 3^{n+1}$$

6. Ejercicios Propuestos

1. Resuelve la relación de recurrencia $a_n = 4a_{n-1} - 4a_{n-2}$ con condiciones iniciales $a_0 = 2$ y $a_1 = 8$. Identifica qué caso de raíces del polinomio característico se presenta.
2. Determina la fórmula de recurrencia no homogénea $a_n = 3a_{n-1} + 2n$ con $a_0 = 1$.
3. Investiga el **Teorema Maestro (Master Theorem)** utilizado en el análisis de algoritmos y explica cómo resuelve de forma directa relaciones de recurrencia del tipo $T(n) = aT(n/b) + f(n)$ para algoritmos de división y conquista.

Tema 9: Funciones Generadoras

Una **función generadora** es una herramienta matemática sumamente potente que permite codificar una sucesión infinita de números reales $(a_0, a_1, a_2, \dots)$ como los coeficientes de una serie formal de potencias. El célebre matemático Herbert Wilf la definió de forma poética: *"Una función generadora es un tendedero en el que colgamos una sucesión de números para exhibirlos"*. En informática, se emplean para modelar problemas de recuento complejos, particiones de memoria y para resolver de forma elegante relaciones de recurrencia.

1. Definición de Función Generadora Ordinaria (FGO)

Dada una sucesión $(a_n)_{n \geq 0} = (a_0, a_1, a_2, \dots)$, su **Función Generadora Ordinaria** es la serie de potencias formal:

$$A(x) = \sum_{n=0}^{\infty} a_n x^n = a_0 + a_1 x + a_2 x^2 + a_3 x^3 + \dots$$

donde x es una variable auxiliar abstracta (no nos preocupa su convergencia numérica, sino la manipulación algebraica de sus coeficientes).

Series de Potencias Fundamentales

Para trabajar con funciones generadoras, asociamos funciones racionales compactas con sus correspondientes desarrollos infinitos:

Función Rational $A(x)$	Serie de Potencias Asociada	Sucesión Generada (a_n)
$\frac{1}{1-x}$	$1 + x + x^2 + x^3 + \dots$	$(1, 1, 1, 1, \dots)$
$\frac{1}{1-ax}$	$1 + ax + a^2 x^2 + a^3 x^3 + \dots$	$(1, a, a^2, a^3, \dots)$
$\frac{1}{(1-x)^2}$	$1 + 2x + 3x^2 + 4x^3 + \dots$	$(1, 2, 3, 4, \dots)$

Función Rational $A(x)$	Serie de Potencias Asociada	Sucesión Generada (a_n)
$(1+x)^k$	$\sum_{n=0}^k \binom{k}{n} x^n$	Coeficientes binomiales $\binom{k}{n}$

2. Operaciones Con Funciones Generadoras

Si $A(x) = \sum a_n x^n$ y $B(x) = \sum b_n x^n$:

1. Suma (Combinación Lineal):

$$\alpha A(x) + \beta B(x) = \sum_{n=0}^{\infty} (\alpha a_n + \beta b_n) x^n$$

2. Desplazamiento a la derecha:

$$x^k A(x) = \sum_{n=0}^{\infty} a_n x^{n+k} = a_0 x^k + a_1 x^{k+1} + \dots \Rightarrow \text{genera } (\underbrace{0, \dots, 0}_{k \text{ veces}}, a_0, a_1, \dots)$$

3. Derivación (Multiplicación por el índice):

$$A'(x) = \sum_{n=1}^{\infty} n a_n x^{n-1} \Rightarrow x A'(x) = \sum_{n=0}^{\infty} n a_n x^n \Rightarrow \text{genera } (0, a_1, 2a_2, 3a_3, \dots)$$

3. Resolución de Recurrencias mediante Funciones Generadoras

El método general consta de 4 pasos lógicos:

- Definir la serie:** Expresar la función generadora $A(x) = \sum_{n=0}^{\infty} a_n x^n$.
- Multiplicar y Sumar:** Multiplicar la ecuación de recurrencia por x^n y sumar desde el orden de la relación hasta el infinito ($\sum_{n=k}^{\infty}$).
- Sustituir y Despejar:** Expresar los sumatorios en términos de la función $A(x)$ y las condiciones iniciales, despejando algebraicamente la función $A(x)$.

4. **Descomponer y Expandir:** Descomponer la función racional en fracciones simples y reescribirlas como series de potencias conocidas para extraer el coeficiente general a_n .
-

4. El Toque Informático

Estructuras de Datos Balanceadas y los Números de Catalán

Un problema clásico en informática es determinar cuántos **árboles binarios de búsqueda distintos** se pueden construir utilizando exactamente n nodos con claves únicas.

- Si llamamos b_n a este número, podemos definir una relación de recurrencia en función del tamaño del subárbol izquierdo (k nodos) y del derecho ($n - 1 - k$ nodos):

$$b_n = \sum_{k=0}^{n-1} b_k \cdot b_{n-1-k} \quad (\text{con } b_0 = 1)$$

- Esta convolución cuadrática se resuelve utilizando funciones generadoras resultando en la ecuación cuadrática $xB(x)^2 - B(x) + 1 = 0$.
- La expansión de esta función da como resultado la famosa sucesión de los **Números de Catalán**:

$$C_n = \frac{1}{n+1} \binom{2n}{n}$$

Saber que existen C_n árboles posibles permite diseñar algoritmos de optimización para equilibrar árboles AVL o Red-Black minimizando el coste de búsqueda $O(\log n)$ en memoria.

A continuación, implementamos en Python una simulación utilizando cálculo simbólico con la librería `sympy` (si está instalada, o una aproximación con derivadas numéricas) para expandir una función generadora y extraer sus coeficientes.

```

import sympy as sp

# Definimos la variable simbólica x
x = sp.Symbol('x')

# Definimos la función generadora racional: A(x) = 1 / (1 - 3*x) + 2 / (1 - x)
A = 1 / (1 - 3*x) + 2 / (1 - x)

# Realizamos la expansión en serie de Taylor alrededor de x = 0 hasta el grado 10
expansion = sp.series(A, x, 0, 10)
print("Expansión en serie de Taylor de A(x):")
print(expansion)

# Extraemos los coeficientes individuales de la serie para verificar la sucesión
print("\nSucesión de coeficientes generada (primeros 7 términos):")
for n in range(7):
    # Extraemos el coeficiente de x^n
    coeff = A.diff(x, n).subs(x, 0) // sp.factorial(n)
    print(f" a_{n} = {coeff}")

```

5. Ejercicios Resueltos

Ejercicio 1

Dada la función generadora:

$$A(x) = \frac{1}{1-3x} + \frac{2}{1-x}$$

Determinar el término general de la sucesión (a_n) que genera.

Solución:

- Separar los términos:** Tenemos $A(x) = A_1(x) + A_2(x)$, donde $A_1(x) = \frac{1}{1-3x}$ y $A_2(x) = 2 \cdot \frac{1}{1-x}$.
- Identificar las series elementales:**
 - $\frac{1}{1-3x} = \sum_{n=0}^{\infty} (3^n)x^n$ (sucesión asociada: 3^n).
 - $2 \cdot \frac{1}{1-x} = 2 \cdot \sum_{n=0}^{\infty} (1^n)x^n = \sum_{n=0}^{\infty} 2x^n$ (sucesión asociada: constante 2).
- Combinar coeficientes:** Sumando los coeficientes de las potencias correspondientes de x^n :

$$a_n = 3^n + 2 \quad (\text{para } n \geq 0)$$

La sucesión generada es: $(3, 5, 11, 29, 83, \dots)$.

Ejercicio 2

Resolver por medio de funciones generadoras la relación de recurrencia:

$$a_n = 2a_{n-1} \quad (\text{para } n \geq 1) \quad \text{con } a_0 = 1$$

Solución:

1. Definir la función generadora:

$$A(x) = \sum_{n=0}^{\infty} a_n x^n = a_0 + \sum_{n=1}^{\infty} a_n x^n$$

2. Multiplicar la relación por x^n y sumar desde $n = 1$:

$$\sum_{n=1}^{\infty} a_n x^n = \sum_{n=1}^{\infty} 2a_{n-1} x^n$$

3. Reescribir los términos en función de $A(x)$:

- Lado izquierdo: $\sum_{n=1}^{\infty} a_n x^n = A(x) - a_0 = A(x) - 1$.
- Lado derecho: sacamos constantes y x fuera del sumatorio para alinear los índices:

$$\sum_{n=1}^{\infty} 2a_{n-1} x^n = 2x \sum_{n=1}^{\infty} a_{n-1} x^{n-1} = 2x \sum_{m=0}^{\infty} a_m x^m = 2xA(x)$$

4. Igualar y despejar $A(x)$:

$$A(x) - 1 = 2xA(x) \quad \Rightarrow \quad A(x)(1 - 2x) = 1 \quad \Rightarrow \quad A(x) = \frac{1}{1 - 2x}$$

5. Expandir de vuelta: Sabemos que la expansión de $\frac{1}{1-2x}$ es $\sum_{n=0}^{\infty} 2^n x^n$.
Extrayendo el coeficiente de x^n :

$$a_n = 2^n$$

6. Ejercicios Propuestos

1. Halla la función generadora en forma racional compacta para la sucesión $(1, 3, 9, 27, 81, \dots)$.
2. Encuentra los primeros 4 coeficientes de la función generadora $A(x) = \frac{1}{(1-x)^3}$ mediante derivación sucesiva.
3. Utiliza el método de funciones generadoras para resolver la relación de recurrencia $a_n = 3a_{n-1} + 2$ con condición inicial $a_0 = 0$.

Tema 10: Conceptos Básicos de Grafos y Árboles

La teoría de grafos es la rama de las matemáticas discretas encargada de modelar y analizar relaciones binarias entre objetos de un conjunto. En informática, casi cualquier estructura relacional (desde la red de enlaces de Internet hasta el mapa de dependencias de un compilador o una red social) se abstrae y modela físicamente en memoria como un **grafo**.

1. Definición Formal y Clasificación

Un **grafo** G es un par ordenado:

$$G = (V, E)$$

donde:

- V : Conjunto no vacío de **vértices** (o nodos).
- E : Conjunto de **aristas** (o arcos), que representan enlaces entre pares de vértices.

Tipos de Grafos

- **Grafo No Dirigido**: Las aristas son conjuntos desordenados de dos vértices $\{u, v\}$. La relación es bidireccional (si u está conectado con v , v está conectado con u).
- **Grafo Dirigido (Dígrafo)**: Las aristas son pares ordenados (u, v) , representados mediante flechas que apuntan de un origen a un destino.
- **Grafo Valorado (Ponderado)**: Cada arista tiene un peso o coste numérico asociado (por ejemplo, distancia en kilómetros o latencia de red en milisegundos).



2. Grado de un Vértice y el Lema del Apretón de Manos

El **grado** de un vértice v , denotado como $\deg(v)$, es el número de aristas incidentes en él.

- En grafos dirigidos, se divide en **grado de entrada** ($\deg^-(v)$) y **grado de salida** ($\deg^+(v)$).

Teorema del Apretón de Manos (Handshaking Lemma)

En cualquier grafo no dirigido, la suma de los grados de todos sus vértices es exactamente igual al doble del número de aristas:

$$\sum_{v \in V} \deg(v) = 2|E|$$

Consecuencia: Todo grafo tiene un número par de vértices con grado impar.

3. Caminos, Conectividad y Árboles

- Camino:** Secuencia de vértices unidos consecutivamente por aristas.
- Ciclo:** Camino cerrado que empieza y termina en el mismo vértice sin repetir aristas ni vértices intermedios.
- Grafo Conexo:** Un grafo no dirigido es conexo si existe un camino entre cualquier par de vértices.

Árboles

Un **árbol** es un grafo no dirigido conexo que **no contiene ciclos**.

- **Propiedad fundamental:** Si un árbol tiene $|V|$ vértices, entonces tiene exactamente $|E| = |V| - 1$ aristas. La eliminación de cualquier arista desconecta el árbol; la adición de cualquier arista nueva crea un ciclo.
-

4. Recorridos sobre Grafos: BFS y DFS

Para procesar o buscar información en un grafo de forma sistemática, se usan dos estrategias fundamentales de recorrido:

1. Búsqueda en Anchura (BFS - Breadth-First Search):

- Visita los nodos nivel a nivel (primero los vecinos directos, luego los vecinos de los vecinos).
- Utiliza una **cola (FIFO)** como estructura de datos auxiliar.
- Es óptimo para hallar el camino con menor número de aristas en grafos no valorados.

2. Búsqueda en Profundidad (DFS - Depth-First Search):

- Sigue un camino explorando lo más profundo posible antes de realizar backtracking (retroceder).
 - Utiliza una **pila (LIFO)** o recursión nativa del procesador.
-

5. El Toque Informático

Redes de Datos, el Algoritmo PageRank y Bases de Datos NoSQL de Grafos

1. **PageRank de Google:** El motor de búsqueda original de Google modela toda la Web como un dígrafo gigante donde los nodos son páginas web y las aristas son hipervínculos. El algoritmo calcula la relevancia de una página midiendo la probabilidad de que un usuario aleatorio acabe en ella, aplicando cadenas de Markov sobre el dígrafo de la Web.
2. **Bases de Datos de Grafos (Neo4j):** Las bases de datos relacionales tradicionales (SQL) sufren caídas de rendimiento drásticas al realizar consultas

cruzadas complejas (JOINS) en redes interconectadas (como redes sociales o sistemas de recomendación). Las bases de datos de grafos almacenan directamente los nodos y punteros físicos a sus aristas en disco, permitiendo realizar búsquedas de relaciones en tiempo constante.

A continuación, implementamos en Python una estructura de grafo no dirigido y los algoritmos de recorrido BFS y DFS en consola.

```
from collections import deque

class Grafo:
    def __init__(self):
        # Representación mediante lista de adyacencia (diccionario)
        self.adj = {}

    def agregar_vertice(self, v):
        if v not in self.adj:
            self.adj[v] = []

    def agregar_arista(self, u, v):
        self.agregar_vertice(u)
        self.agregar_vertice(v)
        self.adj[u].append(v)
        self.adj[v].append(u) # Bidireccional

    # Recorrido BFS (Búsqueda en Anchura)
    def bfs(self, inicio):
        visitados = set([inicio])
        cola = deque([inicio])
        resultado = []

        while cola:
            v = cola.popleft()
            resultado.append(v)
            for vecino in self.adj[v]:
                if vecino not in visitados:
                    visitados.add(vecino)
                    cola.append(vecino)
        return resultado

    # Recorrido DFS (Búsqueda en Profundidad)
    def dfs(self, inicio, visitados=None, resultado=None):
        if visitados is None:
            visitados = set()
        if resultado is None:
            resultado = []

        visitados.add(inicio)
        resultado.append(inicio)
        for vecino in self.adj[inicio]:
            if vecino not in visitados:
                self.dfs(vecino, visitados, resultado)
        return resultado
```

```
# Construimos un grafo de prueba
g = Grafo()
g.agregar_arista('A', 'B')
g.agregar_arista('A', 'C')
g.agregar_arista('B', 'D')
g.agregar_arista('C', 'E')

print("Recorrido BFS empezando en 'A':", g.bfs('A'))
print("Recorrido DFS empezando en 'A':", g.dfs('A'))
```

6. Ejercicios Resueltos

Ejercicio 1

Demostrar formalmente que en cualquier grafo no dirigido, el número de vértices que tienen grado impar es par.

Solución:

1. Partimos del Lema del Apretón de Manos:

$$\sum_{v \in V} \deg(v) = 2|E|$$

2. Dividimos el conjunto de vértices V en dos subgrupos disjuntos: el conjunto V_{par} (vértices de grado par) y el conjunto V_{impar} (vértices de grado impar):

$$\sum_{v \in V_{\text{par}}} \deg(v) + \sum_{v \in V_{\text{impar}}} \deg(v) = 2|E|$$

3. Analizamos los sumandos:

- El término de la derecha $2|E|$ es siempre un número par (por ser múltiplo de 2).
- La primera suma $\sum_{v \in V_{\text{par}}} \deg(v)$ es par, pues sumamos únicamente números pares.

4. Despejamos la segunda suma:

$$\sum_{v \in V_{\text{impar}}} \deg(v) = \underbrace{2|E|}_{\text{Par}} - \underbrace{\sum_{v \in V_{\text{par}}} \deg(v)}_{\text{Par}} \Rightarrow \text{Resultado Par}$$

5. Como cada término individual dentro de la suma $\sum_{v \in V_{\text{impar}}} \deg(v)$ es un número impar, la única forma en que una suma de números impares dé un resultado par es que sumemos un **número par de términos**.

Por lo tanto, el cardinal del conjunto V_{impar} debe ser par. Q.E.D.

Ejercicio 2

Un grafo tiene 6 vértices con los siguientes grados: $\{2, 2, 3, 4, 3, 2\}$. Determinar el número de aristas del grafo.

Solución: Aplicamos el Lema del Apretón de Manos de forma directa:

1. Sumamos los grados de todos los vértices:

$$\sum \deg(v) = 2 + 2 + 3 + 4 + 3 + 2 = 16$$

2. Igualamos a dos veces el número de aristas:

$$2|E| = 16 \quad \Rightarrow \quad |E| = \frac{16}{2} = 8$$

El grafo tiene exactamente 8 aristas.

7. Ejercicios Propuestos

1. Dibuja un grafo conexo de 5 vértices que sea euleriano (tenga un ciclo euleriano) y otro de 5 vértices que no lo sea, explicando en base a los grados de los vértices la diferencia.
2. Demuestra que todo árbol con más de un vértice tiene al menos dos vértices colgantes (hojas, es decir, de grado 1).
3. Explica la diferencia entre las estructuras de datos cola (FIFO) y pila (LIFO) y cómo influyen en el comportamiento del recorrido de un grafo (BFS vs DFS).

Tema 11: Representación Matricial de Grafos

Para procesar grafos en computadores, es necesario traducir las estructuras visuales de vértices y aristas en estructuras de datos de memoria eficientes. Las dos formas principales de codificación matemática y digital son las **Matrices de Adyacencia** y las **Listas de Adyacencia**. La matriz de adyacencia, en particular, conecta de forma brillante la teoría de grafos con el álgebra lineal, permitiendo resolver problemas combinatorios de caminos mediante simples multiplicaciones de matrices.

1. Métodos de Representación en Computadores

1.1 Listas de Adyacencia

Asocian a cada vértice un vector dinámico o lista enlazada conteniendo sus vecinos directos.

- **Espacio en memoria:** $O(|V| + |E|)$.
- **Uso ideal:** Para **grafos dispersos (sparse graphs)**, donde el número de aristas es mucho menor que el máximo posible ($|E| \ll |V|^2$). La inmensa mayoría de las redes del mundo real (como los enlaces web o redes eléctricas) son dispersas.

1.2 Matriz de Adyacencia

Es una matriz cuadrada A de dimensiones $|V| \times |V|$ donde las filas y columnas corresponden a los vértices del grafo.

$$A_{ij} = \begin{cases} 1 & \text{si existe una arista entre } v_i \text{ y } v_j \\ 0 & \text{en caso contrario} \end{cases}$$

- **Espacio en memoria:** $O(|V|^2)$, independientemente del número de aristas.

- **Uso ideal:** Para **grafos densos (dense graphs)** ($|E| \approx |V|^2$), o cuando se requiere verificar en tiempo constante $O(1)$ si existe un enlace entre dos nodos.

Grafo	Matriz de Adyacencia
(1)---(2) / / (3) (4)	1 2 3 4 1 [0 1 1 0] 2 [1 0 1 0] 3 [1 1 0 0] 4 [0 0 0 0]

- **Propiedades (Grafos No Dirigidos):**
 - La matriz A es **simétrica** respecto a la diagonal principal ($A = A^T$).
 - Si el grafo no contiene bucles (lazos de un nodo a sí mismo), la diagonal principal se compone exclusivamente de ceros.
 - La suma de los elementos de la fila i es igual al grado del vértice v_i .

2. El Teorema de Caminos de Longitud k

Una de las aplicaciones más sorprendentes de la representación matricial es el cálculo del número de caminos entre nodos utilizando potencias de matrices.

Teorema

Sea A la matriz de adyacencia de un grafo G . El elemento situado en la fila i y columna j de la matriz potencia A^k (donde $k \in \mathbb{Z}^+$):

$$(A^k)_{ij}$$

es igual al **número de caminos distintos de longitud exactamente k** que existen entre el vértice v_i y el vértice v_j .

3. Matriz de Incidencia

Es una matriz M de dimensiones $|V| \times |E|$ (filas vértices, columnas aristas).

- En un grafo no dirigido:

$$M_{ij} = \begin{cases} 1 & \text{si el v\u00e9rtice } v_i \text{ es incidente en la arista } e_j \\ 0 & \text{en caso contrario} \end{cases}$$

- Cada columna tiene exactamente dos valores 1, correspondientes a los dos v\u00e9rtices extremos que conecta la arista.

4. El Toque Inform\u00e1tico

Rendimiento de Algoritmos seg\u00fan la Representaci\u00f3n (Trade-offs)

En programaci\u00f3n, elegir la representaci\u00f3n correcta impacta cr\u00edticamente el rendimiento de las operaciones b\u00e1sicas:

Operaci\u00f3n	Matriz de Adyacencia	Lista de Adyacencia
Espacio en Memoria	$\mathcal{O}(V^2)$	$\mathcal{O}(V)$
\u00bfExiste arista (u, v) ?	$\mathcal{O}(1)$	$\mathcal{O}(\text{deg}(u))$
Listar vecinos de u	$\mathcal{O}(V)$	$\mathcal{O}(\text{deg}(u))$
Insertar/Eliminar arista	$\mathcal{O}(V)$	$\mathcal{O}(\text{deg}(u))$

Por ejemplo, si un grafo modela una red de 100.000 usuarios (nodos), una matriz de adyacencia exigir\u00eda almacenar $100.000^2 = 10^{10}$ enteros (≈ 40 GB de RAM). Si cada usuario tiene de media solo 50 amigos, una lista de adyacencia requerir\u00e1 \u00fanicamente almacenar los punteros de $50 \times 100.000 = 5 \times 10^6$ conexiones (≈ 20 MB de memoria), permitiendo que quepa en la cach\u00e9 del procesador.

A continuaci\u00f3n, implementamos en Python una simulaci\u00f3n utilizando la librer\u00eda `numpy` para comprobar el Teorema de Caminos de Longitud k .

```
import numpy as np

# Definimos la matriz de adyacencia de un grafo de 4 v\u00e9rtices
# 1-2, 1-3, 2-3, 3-4 (V\u00e9rtices indexados de 0 a 3)
A = np.array([
    [0, 1, 1, 0],
    [1, 0, 1, 0],
    [1, 1, 0, 1],
    [0, 0, 1, 0]
```

```

    [0, 0, 1, 0]
])

print("Matriz de Adyacencia A (Caminos de longitud 1):")
print(A)

# Calculamos A^2 (Caminos de longitud 2)
A_2 = np.linalg.matrix_power(A, 2)
print("\nMatriz A^2 (Caminos de longitud 2):")
print(A_2)

# Calculamos A^3 (Caminos de longitud 3)
A_3 = np.linalg.matrix_power(A, 3)
print("\nMatriz A^3 (Caminos de longitud 3):")
print(A_3)

# Verificación de caminos de longitud 3 entre nodo 0 y nodo 3:
n_caminos = A_3[0, 3]
print(f"\nNúmero de caminos de longitud 3 entre nodo 0 y nodo 3: {n_caminos}")
print("Caminos físicos equivalentes a comprobar: (0->1->2->3) y (0->2->1->2)
[inválido, contiene lazo], etc.")

```

5. Ejercicios Resueltos

Ejercicio 1

Dado el grafo no dirigido compuesto por 3 vértices conectados en línea (1 – 2 – 3):

1. Escribir su matriz de adyacencia A .
2. Calcular A^2 y determinar cuántos caminos de longitud 2 existen entre el nodo 1 y el nodo 3.

Solución:

1. **Construir la matriz de adyacencia A :** Los enlaces son $\{1, 2\}$ y $\{2, 3\}$.

$$A = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix}$$

2. **Calcular la matriz potencia A^2 :** Multiplicamos la matriz por sí misma:

$$A^2 = A \cdot A = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix} \cdot \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix}$$

Calculamos los componentes de la primera fila:

- Fila 1, Columna 1: $(0 \cdot 0 + 1 \cdot 1 + 0 \cdot 0) = 1$.
- Fila 1, Columna 2: $(0 \cdot 1 + 1 \cdot 0 + 0 \cdot 1) = 0$.
- Fila 1, Columna 3: $(0 \cdot 0 + 1 \cdot 1 + 0 \cdot 0) = 1$.

Siguiendo el producto para el resto de las filas resulta:

$$A^2 = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 2 & 0 \\ 1 & 0 & 1 \end{pmatrix}$$

3. **Identificar el número de caminos:** El elemento $(A^2)_{1,3}$ está en la fila 1 y columna 3, y su valor es **1**. Esto significa que existe exactamente **1 camino** de longitud 2 entre el nodo 1 y el nodo 3. *Verificación física:* El único camino de longitud 2 es el camino $1 \rightarrow 2 \rightarrow 3$.
-

6. Ejercicios Propuestos

1. Dibuja el grafo dirigido correspondiente a la siguiente matriz de adyacencia:

$$A = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 2 \\ 1 & 0 & 0 \end{pmatrix}$$

(Nota: Los valores mayores a 1 indican multígrafos o aristas paralelas).

2. Deduce algebraicamente por qué en cualquier matriz de adyacencia de un grafo no dirigido, el número de elementos con valor "1" en toda la matriz es igual a $2|E|$.
3. Escribe un pseudocódigo o función en C++ para listar todos los vecinos de un nodo u utilizando una representación basada en matrices de adyacencia frente a una representación de listas de adyacencia, comparando sus costes temporales asociados.

Tema 12: Algoritmos sobre Grafos

La resolución práctica de problemas complejos en informática (como calcular la ruta óptima de reparto de un mensajero, trazar la red de fibra óptica de una ciudad o enrutar paquetes por Internet) exige el uso de algoritmos clásicos sobre grafos. En este tema, abordamos los dos problemas algorítmicos fundamentales en grafos ponderados: el cálculo de los **Caminos Mínimos** (Algoritmo de Dijkstra) y la construcción de los **Árboles de Expansión Mínima** (Algoritmos de Prim y Kruskal).

1. El Problema de los Caminos Mínimos: Algoritmo de Dijkstra

Dado un grafo ponderado con pesos no negativos y un vértice de origen $s \in V$, el **Algoritmo de Dijkstra** determina el camino de coste mínimo desde s hasta todos los demás vértices del grafo.

Principio de Funcionamiento (Codicioso / Greedy)

1. Mantiene una estimación de la distancia mínima $d[v]$ para cada vértice. Inicialmente $d[s] = 0$ y $d[v] = \infty$ para los demás.
2. Marca todos los nodos como no visitados y los inserta en una **cola de prioridad** ordenada por distancia mínima estimada.
3. En cada paso, extrae el vértice u no visitado con menor distancia $d[u]$ (comportamiento codicioso).
4. **Relajación de aristas:** Para cada vecino v de u , comprueba si el camino a través de u es más corto que la distancia estimada actual:

$$\text{Si } d[u] + w(u, v) < d[v] \quad \Rightarrow \quad d[v] = d[u] + w(u, v)$$

5. Repite el proceso hasta vaciar la cola.

- **Complejidad:** Implementado con una cola de prioridad basada en montículos binarios (Min-Heap), su coste es de $O((|V| + |E|) \log |V|)$.
-

2. Árboles de Expansión Mínima (MST)

Dado un grafo no dirigido conexo y ponderado, un **Árbol de Expansión Mínima (MST)** es un subgrafo que contiene a todos los vértices de la red (V), es un árbol (conexo y acíclico) y la **suma de los pesos de sus aristas es la mínima posible**.

2.1 Algoritmo de Prim

Comienza desde un nodo raíz inicial y va haciendo crecer el árbol nodo a nodo. En cada paso, selecciona la arista de menor peso que conecta un nodo ya perteneciente al árbol con un nodo externo. Es muy eficiente en grafos densos utilizando colas de prioridad ($O(|E| \log |V|)$).

2.2 Algoritmo de Kruskal

Funciona ordenando las aristas del grafo de menor a mayor peso y seleccionando aristas individualmente de forma codiciosa:

1. Ordena el conjunto de aristas E por peso ascendente.
2. Crea un bosque inicial donde cada vértice es un conjunto disjunto aislado.
3. Para cada arista (u, v) en orden:
 - Si u y v pertenecen a componentes conexas distintas (no forman ciclo), **se añade la arista al MST** y se unen (fusionan) ambas componentes.
 - Si pertenecen al mismo conjunto, se descarta la arista (pues añadirla cerraría un ciclo).
4. El algoritmo termina al seleccionar exactamente $|V| - 1$ aristas.

La Estructura Disjoint-Set (Union-Find)

Para verificar rápidamente si dos nodos forman ciclo y unirlos, Kruskal utiliza una estructura de **conjuntos disjuntos (Union-Find)**. Esta estructura reduce el coste de detección a casi tiempo constante $O(\alpha(|V|))$, donde α es la inversa de la función de Ackermann (crecimiento extremadamente lento).

3. El Toque Informático

Enrutamiento de Internet (OSPF) y Minimización de Costes Físicos

1. **OSPF (Open Shortest Path First)**: Es el protocolo de enrutamiento interno más utilizado en los routers de las redes corporativas y proveedores de Internet. Cada router almacena un mapa del dígrafo de la red y ejecuta localmente el **Algoritmo de Dijkstra** para calcular la ruta más rápida (con menor retraso de propagación) para redirigir los paquetes de datos hacia su destino.
2. **Cableado de Redes (MST)**: Si una empresa de telecomunicaciones desea conectar 20 ciudades con fibra óptica subterránea, no necesita cablearlas todas entre sí (grafo completo). Para minimizar los kilómetros de zanja excavada, calcula el **MST** mediante **Kruskal** o **Prim**, garantizando que todas las ciudades queden interconectadas con el mínimo coste total de cableado posible.

A continuación, implementamos en C++ el Algoritmo de Kruskal utilizando la estructura de datos Union-Find para resolver el MST de un grafo ponderado.

```
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

// Estructura para representar una arista ponderada
struct Arista {
    int origen, destino, peso;

    // Sobrecarga del operador menor para ordenar por peso
    bool operator<(const Arista& otra) const {
        return peso < otra.peso;
    }
};

// Estructura de Conjuntos Disjuntos (Disjoint-Set / Union-Find)
struct UnionFind {
    vector<int> padre, rango;

    UnionFind(int n) {
        padre.resize(n);
        rango.resize(n, 0);
        for (int i = 0; i < n; i++) {
            padre[i] = i; // Cada nodo es su propio padre al inicio
        }
    }
};
```

```

}

// Operación Find con compresión de caminos
int buscar(int i) {
    if (padre[i] == i)
        return i;
    return padre[i] = buscar(padre[i]);
}

// Operación Union por rango
void unir(int i, int j) {
    int raiz_i = buscar(i);
    int raiz_j = buscar(j);
    if (raiz_i != raiz_j) {
        if (rango[raiz_i] < rango[raiz_j]) {
            padre[raiz_i] = raiz_j;
        } else if (rango[raiz_i] > rango[raiz_j]) {
            padre[raiz_j] = raiz_i;
        } else {
            padre[raiz_j] = raiz_i;
            rango[raiz_i]++;
        }
    }
}

};

// Función principal del algoritmo de Kruskal
void kruskal(int num_vertices, vector<Arista>& aristas) {
    // 1. Ordenamos las aristas de menor a mayor peso
    sort(aristas.begin(), aristas.end());

    UnionFind uf(num_vertices);
    vector<Arista> mst;
    int peso_total_mst = 0;

    // 2. Procesamos secuencialmente cada arista
    for (const auto& arista : aristas) {
        int conjunto_origen = uf.buscar(arista.origen);
        int conjunto_destino = uf.buscar(arista.destino);

        // Si no pertenecen al mismo conjunto, no forman ciclo
        if (conjunto_origen != conjunto_destino) {
            mst.push_back(arista);
            peso_total_mst += arista.peso;
            uf.unir(conjunto_origen, conjunto_destino);
        }
    }

    // Mostramos el resultado del MST
    cout << "Aristas en el MST:" << endl;
    for (const auto& arista : mst) {
        cout << arista.origen << " - " << arista.destino << " (Peso: " <<
arista.peso << ")" << endl;
    }
    cout << "Peso total del MST: " << peso_total_mst << endl;
}

int main() {

```

```
int vertices = 4;
vector<Arista> aristas = {
    {0, 1, 10},
    {0, 2, 6},
    {0, 3, 5},
    {1, 3, 15},
    {2, 3, 4}
};

kruskal(vertices, aristas);
return 0;
}
```

4. Ejercicios Resueltos

Ejercicio 1

Aplicar el algoritmo de Kruskal paso a paso para hallar el Árbol de Expansión Mínima del siguiente grafo no dirigido:

- Vértices: $\{0, 1, 2, 3\}$.
- Aristas ponderadas: $e_1 = (0, 3)$ peso 5; $e_2 = (2, 3)$ peso 4; $e_3 = (0, 2)$ peso 6; $e_4 = (0, 1)$ peso 10; $e_5 = (1, 3)$ peso 15.

Solución:

1. Ordenar las aristas por peso:

1. $(2, 3)$ - peso 4
2. $(0, 3)$ - peso 5
3. $(0, 2)$ - peso 6
4. $(0, 1)$ - peso 10
5. $(1, 3)$ - peso 15

2. Inicializar conjuntos disjuntos: $\{0\}, \{1\}, \{2\}, \{3\}$.

3. Procesar aristas secuencialmente:

- Arista 1: $(2, 3)$, peso 4. Vértice 2 y 3 están en conjuntos distintos.
Seleccionamos arista. Unimos conjuntos: $\{0\}, \{1\}, \{2, 3\}$. MST actual: $\{(2, 3)\}$.
- Arista 2: $(0, 3)$, peso 5. Vértice 0 y 3 están en conjuntos distintos.
Seleccionamos arista. Unimos conjuntos: $\{1\}, \{0, 2, 3\}$. MST actual: $\{(2, 3), (0, 3)\}$.

- Arista 3: $(0, 2)$, peso 6. Vértices 0 y 2 ya están en el mismo conjunto ($\{0, 2, 3\}$). Añadirla crearía el ciclo $(0 - 2 - 3 - 0)$. **Descartamos arista.**
- Arista 4: $(0, 1)$, peso 10. Vértice 0 y 1 están en conjuntos distintos. **Seleccionamos arista.** Unimos conjuntos: $\{0, 1, 2, 3\}$. MST actual: $\{(2, 3), (0, 3), (0, 1)\}$.

4. **Finalización:** Hemos seleccionado 3 aristas ($|V| - 1 = 3$). Las componentes están unificadas.

- Peso total del MST = $4 + 5 + 10 = 19$.

Ejercicio 2

Un algoritmo de Dijkstra se ejecuta en un router. Explica por qué el algoritmo no funcionaría de forma correcta si una de las líneas de comunicación tuviera un coste negativo (por ejemplo, $w(u, v) = -5$).

Solución: Dijkstra es un algoritmo de naturaleza **codiciosa (greedy)**. Una vez que selecciona un nodo con la menor distancia provisional y lo marca como visitado, asume que su distancia mínima es definitiva y **nunca vuelve a evaluarlo**. Si existen aristas de peso negativo, este principio falla: podría ocurrir que un camino posterior que pase por dicha arista negativa reduzca el coste de un nodo ya considerado óptimo, requiriendo reevaluar nodos cerrados. Para grafos con pesos negativos, se debe utilizar el **Algoritmo de Bellman-Ford** (con complejidad superior $O(|V||E|)$) o el algoritmo de Floyd-Warshall.

5. Ejercicios Propuestos

1. Dibuja un grafo ponderado de 5 vértices y aplica paso a paso el algoritmo de Dijkstra para calcular los caminos mínimos partiendo desde el vértice 0.
2. Explica la diferencia de enfoque entre el algoritmo de Prim y el de Kruskal para hallar el MST. ¿Bajo qué circunstancias de densidad de red es más eficiente implementar cada uno de ellos?
3. ¿Cómo funciona la técnica de **Compresión de Caminos (Path Compression)** en el método Union-Find y cómo reduce la complejidad temporal en la detección de ciclos de Kruskal?

Glosario de Términos

- **Algoritmo de Dijkstra:** Algoritmo codicioso (greedy) que calcula el camino más corto desde un nodo origen a todos los demás de un grafo ponderado con pesos no negativos.
- **Árbol:** Grafo no dirigido conexo que no contiene ciclos. Propiedad fundamental: $|E| = |V| - 1$.
- **Árbol de Expansión Mínima (MST):** Subgrafo recubridor conexo y acíclico de un grafo ponderado que conecta todos los vértices con el menor peso total acumulado.
- **Complejidad Algorítmica:** Medida abstracta del crecimiento en tiempo de ejecución (complejidad temporal) o consumo de memoria (complejidad espacial) de un algoritmo en función del tamaño de entrada.
- **Criptografía Asimétrica:** Sistema criptográfico que utiliza pares de claves complementarias (pública para cifrar, privada para descifrar) resolviendo el problema de la distribución de claves.
- **Deducción Natural:** Sistema de derivación lógica formal paso a paso, donde cada paso se justifica aplicando una regla de inferencia o equivalencia predefinida.
- **Función Generadora Ordinaria:** Serie de potencias formal cuyos coeficientes contienen los términos de una sucesión matemática. Utilizada para resolver recurrencias y modelar recuentos.
- **Grafo:** Estructura discreta definida por un par ordenado $G = (V, E)$, donde V es el conjunto de vértices y E es el conjunto de aristas.
- **Identidad de Bézout:** Teorema que afirma que existen enteros x e y tales que $a \cdot x + b \cdot y = \text{mcd}(a, b)$, resolviendo los coeficientes del algoritmo extendido de Euclides.
- **Inverso Modular:** Entero x tal que $a \cdot x \equiv 1 \pmod{m}$. Existe si y solo si a y m son primos entre sí ($\text{mcd}(a, m) = 1$).
- **Principio del Palomar:** Principio combinatorio que establece que si n objetos se colocan en m cajas y $n > m$, al menos una caja debe contener más de un objeto.
- **Relación de Recurrencia:** Ecuación algebraica que define de forma recursiva una sucesión numérica en función de sus valores previos.
- **Tautología:** Proposición compuesta que es verdadera para todas las combinaciones posibles de valores de verdad de sus variables constituyentes.

Bibliografía Recomendada

1. **Grimaldi, R. P. (2006).** *Matemáticas discreta y combinatoria* (3ª ed.). Addison-Wesley.
 - *Nota:* Texto de referencia fundamental con una excelente aproximación a relaciones de recurrencia, sumatorias lógicas y combinatoria de recuento.
2. **Rosen, K. H. (2012).** *Discrete Mathematics and its Applications* (7th ed.). McGraw-Hill.
 - *Nota:* Una obra de gran valor pedagógico con aplicaciones prácticas muy claras orientadas al análisis de algoritmos y la ingeniería.
3. **Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009).** *Introduction to Algorithms* (3rd ed.). MIT Press.
 - *Nota:* El manual definitivo para comprender la teoría y la correctitud de los algoritmos de grafos (Dijkstra, Kruskal, Prim) y su complejidad temporal.
4. **Graham, R. L., Knuth, D. E., & Patashnik, O. (1994).** *Concrete Mathematics: A Foundation for Computer Science* (2nd ed.). Addison-Wesley.
 - *Nota:* Un clásico imprescindible para profundizar en herramientas matemáticas avanzadas como las funciones generadoras y relaciones de recurrencia aplicadas al análisis de algoritmos.
5. **Stinson, D. R. (2006).** *Cryptography: Theory and Practice* (3rd ed.). Chapman & Hall/CRC.
 - *Nota:* Excelente manual técnico para comprender el sustento algebraico (aritmética modular y teoría de números) detrás del algoritmo de clave pública RSA.